模式识别

第三章：非线性判别函数

北京航空航天大学 人工智能研究院
Institute of Artificial Intelligence， Beihang University

杨鸿宇

2022年秋季学期

Fall 2022

回顾

- 线性判别函数的基本概念
- Fisher线性判别函数
- 感知准则函数
- 最小平方误差准则函数
- 多类问题

线性分类器设计

- 利用训练样本建立线性判别函数

$$g(x) = \boxed{w^T} x + \boxed{w_0}$$

$$g(x) = w_0 + \sum_{i=1}^{d} w_i x_i = \sum_{i=1}^{d} a_i y_i = \boxed{a^T} y$$

最好的结果一般出现在准则函数的极值点上，所以**将分类器设计问题转化为求准则函数极值** w^*, w_0^* **或** a^* **的问题。**

步骤1： 具有类别标志的样本集$\mathcal{X} = \{x_1, x_2, \dots x_N\}$或其增广样本集$\mathcal{Y}$。

步骤2： 确定准则函数$\mathcal{J}$，满足①$\mathcal{J}$是样本集和w, w_0或a的函数；②$\mathcal{J}$的值反应分类器的性能，其极值对应“最好”的决策。

步骤3： 优化求解准则函数极值w^*, w_0^* 或a^* 。

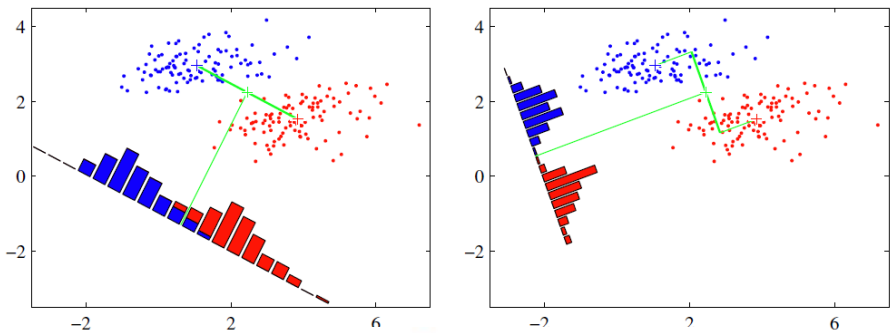
最终得到线性判别函数：$g(x) = w^{*T} x + w_0^*$ 或 $g(x) = a^{*T} y$ ，对于未知类别样本x_k，计算$g(x_k)$并通过决策规则判断其类别。

准则函数

- **Fisher准则**

$$J_F(w) = \frac{(\tilde{m}_1 - \tilde{m}_2)^2}{\tilde{S}_1^2 + \tilde{S}_2^2}$$

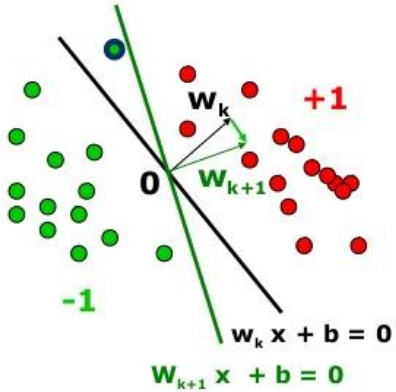
$$w^* = S_w^{-1}(m_1 - m_2)$$



- **感知机准则**

$$J_P(a) = \sum_{y \in \eta^k} (-a^T y)$$

$$a(k+1) = a(k) + \rho_k \nabla \sum_{y \in \eta^k} y$$

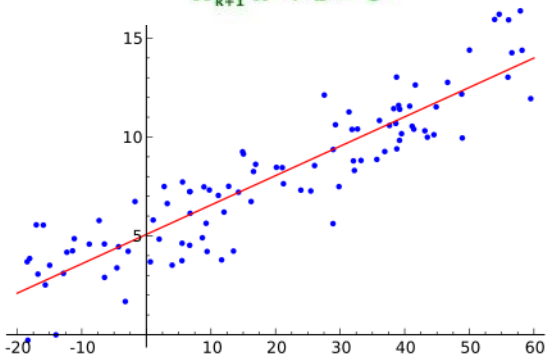


- **最小平方误差准则**

$$J_S(a) = \|e\|^2 = \|Ya - b\|^2 = \sum_{n-1}^{N} (a^T y_n - b_n)^2$$

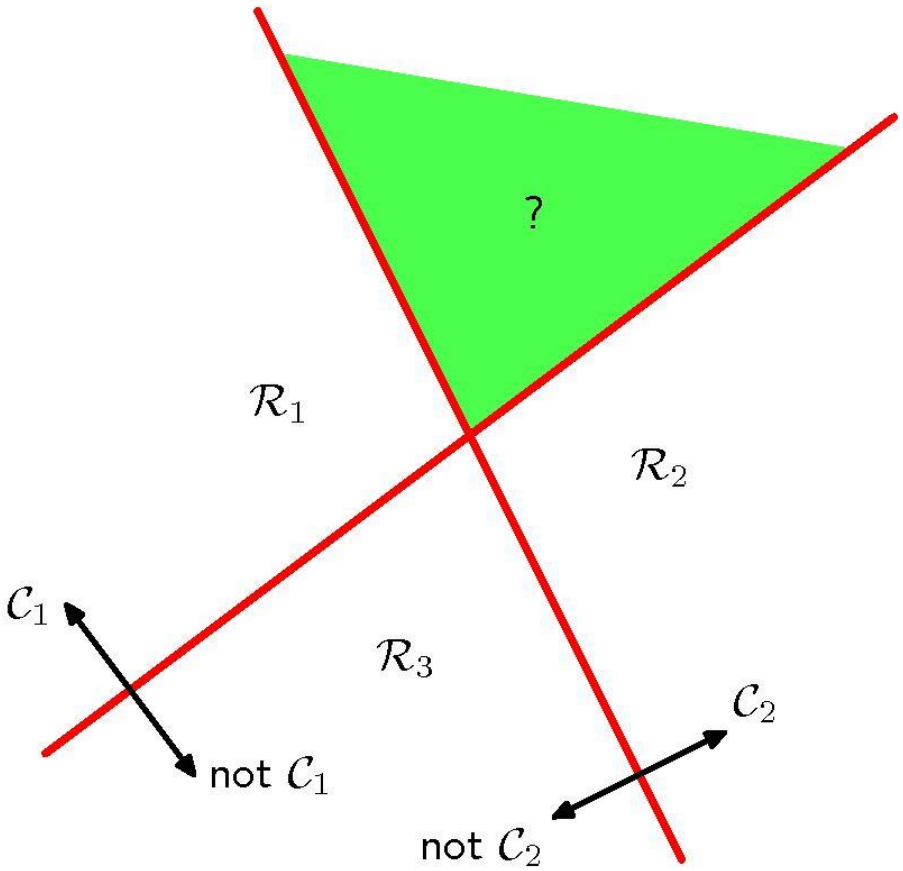
$$a^* = (Y^T Y)^{-1} Y^T b = Y^+ b$$

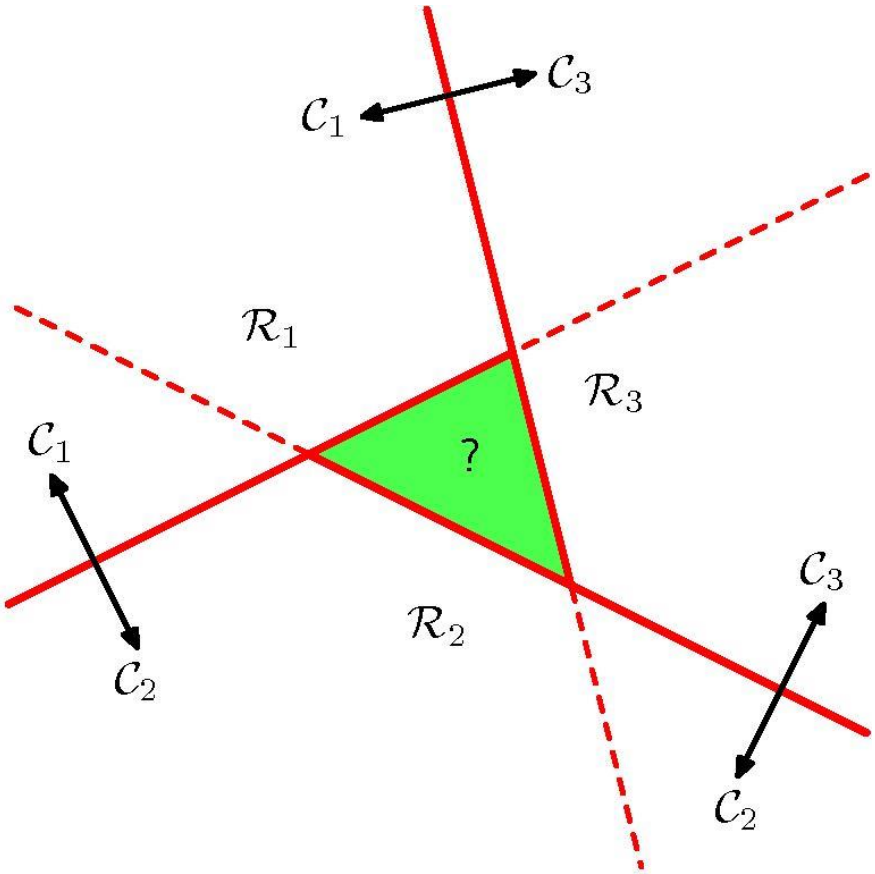
$$a(k+1) = a(k) - \rho_k Y^T (Ya - b)$$



多分类问题

- **1 *vs.* (N-1) or 1 *vs.* 1**





内容

- 引言
- 分段线性判别函数
- 分段线性判别函数的设计
- 二次判别函数
- 神经网络概述
- BP神经网络
- BP神经网络模型改进
- 其他神经网络模型

引言

- 线性判别函数：简单、经济、实用
- 但线性不可分时错误率可能较大

问题线性不可分
- 噪声影响
- 问题本身
  1. 使用新的特征
  2. 非线性变换
  3. 非线性分类器

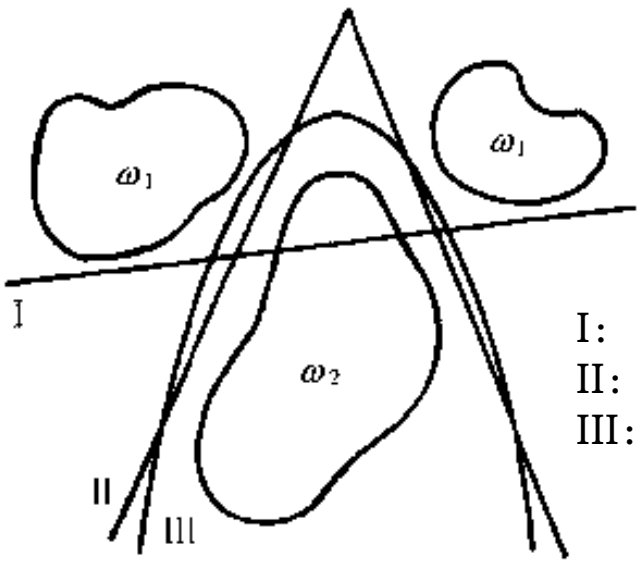
- 实际问题中需要更复杂的**非线性判别函数**来分类
- 注：非线性判别函数并不明确指代一类判别函数，而是指除了线性判别函数之外的各种判别函数！

分段线性判别函数

- 决策面由若干个超平面段组成，计算相对比较简单；
- 能够逼近各种形状的超曲面，适应各种复杂的数据分布情况；
- 实际情况下类别之间的划分无法用解析形式表示时，非线性判别函数仍能够对判别函数进行逼近。

思路：

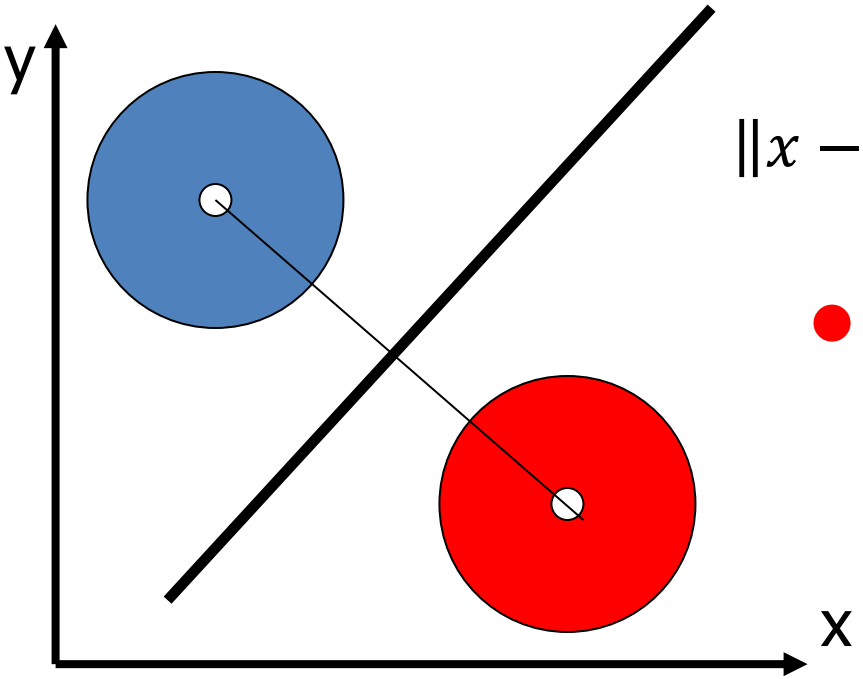
在各类别数据分布式多峰的情况下，可以把**每类划分成为若干子类**，**使得每类是单峰分布并尽可能在各维上对称**。因此，设计多个线性分类器，比较待分类样本与各子类的距离从而实现分段线性分类器。



I： 线性判别
II： 分段线性判别
III： 二次判别

基于距离的分段线性判别函数

- 当两类的类条件概率密度为正态分布且两类先验概率相等，各维特征独立且方差相等时，最小错误率贝叶斯决策是基于最小距离的分段线性判别函数：



$$\|x-\mu_1\|^2-\|x-\mu_2\|^2 \lesseqgtr 0 \rightarrow x \in \begin{cases}\omega_1\\ \omega_2\end{cases}$$

- 在两类情况下，最小距离分类器就是两类均值之间连线的垂直平分面。可以把类均值看做是该类的代表点。

基于距离的分段线性判别函数

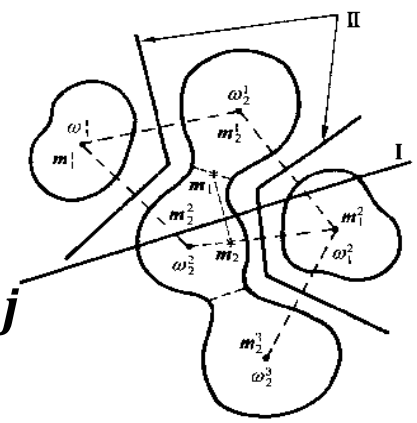
- **分段线性距离分类器**：将各类别划分为相对密集的子类，每个子类以它们的均值作为代表点，然后按最小距离分类。
- 判别函数定义：ω_i有l_i个子类，即属于ω_i的决策域R_i分成l_i个子域$R_i^1, R_i^2, \ldots, R_i^{l_i}$，每个子区域用均值$m_i^k$代表

$$g_i(x) = \min_{k=1,\ldots,l_i} ||x - m_i^k||$$

- 判别规则：

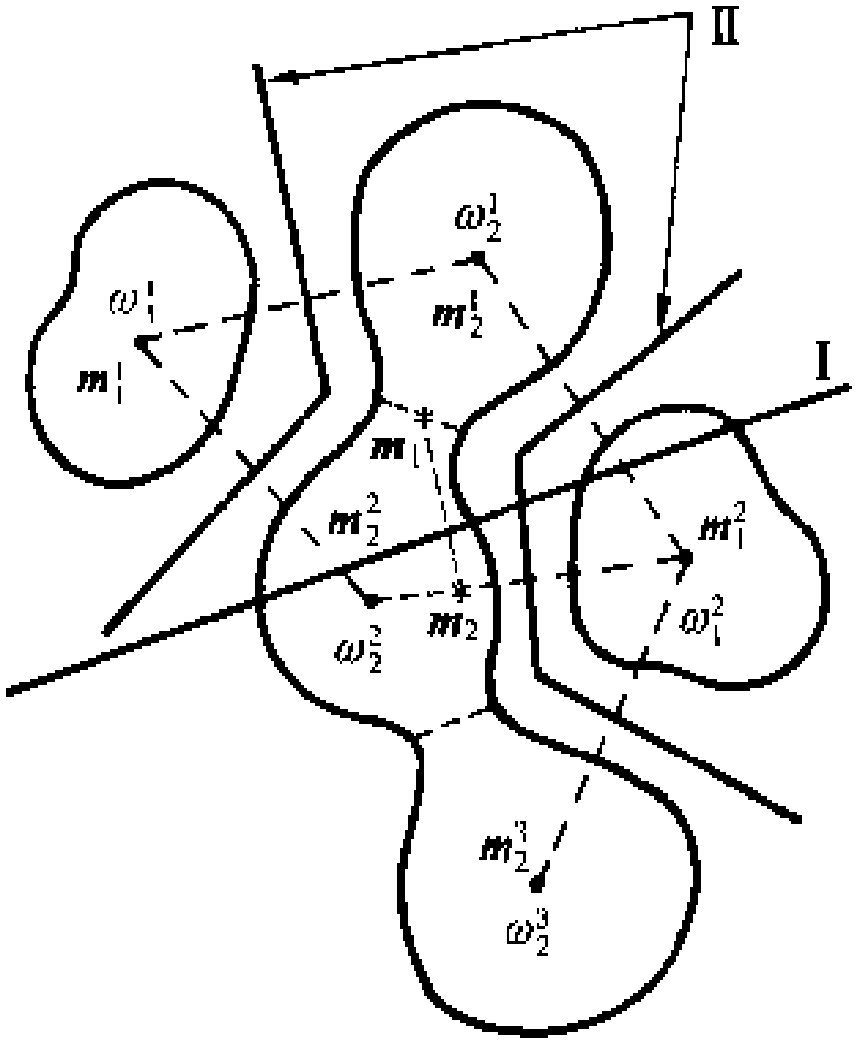
$$j = \arg\min_{i=1,\ldots,c} g_i(x)$$

- 如果$g_j(x) = \arg\min_{i=1,\ldots,c} g_i(x)$则$x$属于$\omega_j$



I： 线性判别
II： 分段线性距离判别

基于距离的分段线性判别函数

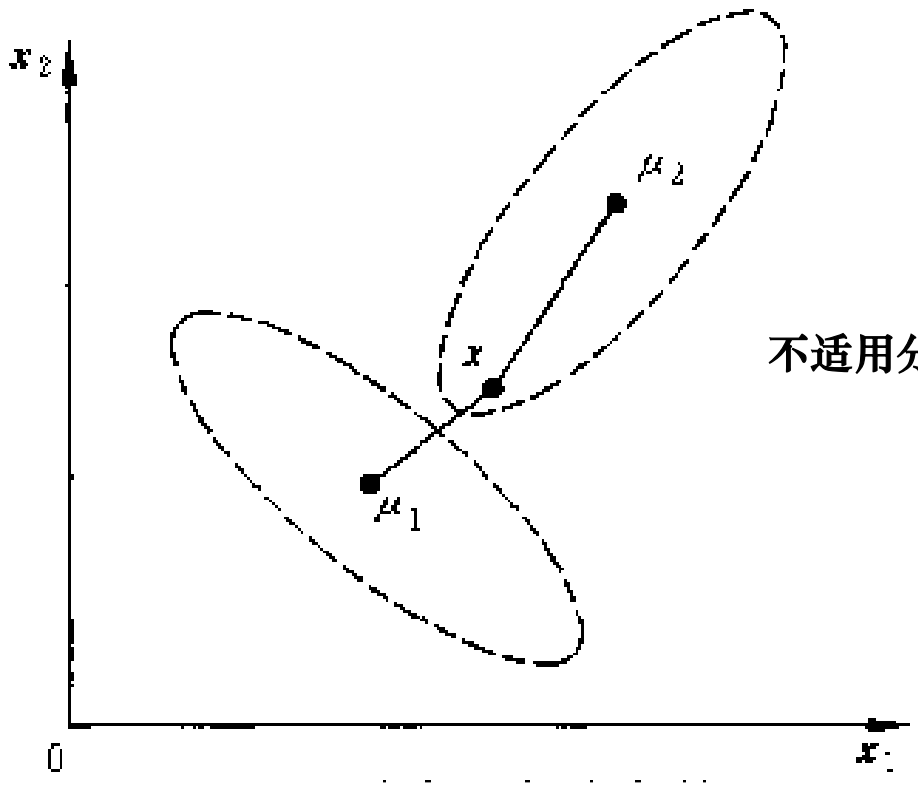


I:　线性判别

II:　分段线性距离判别

一般的分段线性判别函数

- 分段线性距离分类器是较为特殊的情况，适用于各维分布基本对称的情形，对于更复杂的情形则不适用。



不适用分段线性距离分类器

一般的分段线性判别函数

- 建立更一般形式的分段线性判别函数，对于每个子类定义一个线性判别函数：$g_i^k(x)$表示第i类第k段线性判别函数，l_i是第i类所具有的判别函数（子类）个数，w_i^k和w_{i0}^k分别是第k段的权向量与阈值权。$g_i^k(x)$ 可以写为：

$$g_i^k(x) = w_i^{(k)T} x + w_{i0}^k$$

其中，$k = 1,2,\ldots,l_i$，$i = 1,2,\ldots,c$

- 对于权向量和阈值，可以进一步用规范化的增广的形式表示：

$$g_i^k(\mathrm{y}) = \alpha_i^{(k)T} \cdot \mathbf{y}$$

回顾：规范化增广样本向量

- 线性可分性

一组容量为N的样本集$y_1, y_2, \cdots y_N$，其中y_n为$\hat{d}$维增广样本向量，分别来自w_1类和w_2类，如果存在权向量a，使得对于任何$y \in w_1$，都有$a^T y > 0$，而对于任何$y \in w_2$，都有$a^T y < 0$，则称这组样本为线性可分的，反之亦然成立。

$$\begin{cases} a^T y_i > 0, y_i \in w_1 \\ a^T y_j < 0, y_j \in w_2 \end{cases}$$

- 样本的规范化

$$y_n' = \begin{cases} y_i > 0, y_i \in w_1 \\ -y_j < 0, y_j \in w_2 \end{cases}$$

规范化增广样本向量

$$a^T y_n' > 0, n = 1, 2, \ldots, N$$

一般的分段线性判别函数

- 第i类的分段线性判别函数就定义为：

$$g_i(x) = \max_{k=1,2,\ldots,l_i} g_i^k(x)$$

- 决策规则是：

如果$g_j(x) = \arg\max_{i=1,\ldots,c} g_i(x)$，则$x$属于$\omega_j$

一般的分段线性判别函数

- 决策面取决于相邻的决策域，如果第i类的第n个子类与第j类的第m个子类相邻，则由他们共同决定的决策面方程为：

$$g_i^n(x) = g_j^m(x)$$

- 由于$g_i(x)$和$g_j(x)$都是分段线性判别函数，它们之间的决策面也是由多个分段的超平面构成的。
- 其中的一段是某个子类和另一类中相邻子类之间的分类面。

分段线性判别函数的设计

- 问题：
  - 如何确定子类数目？
  - 如何求得各子类的线性判别函数?

- 第一种情况：如果已知子类划分，则可直接使用多类线性分类方法
  - 先验（例如同一种疾病中的病人按照性别、年龄等划分子类）、聚类分析

分段线性判别函数的设计

- 第二种情况：只知道子类数目，不知道子类划分，用下述的错误修正法（此法介绍使用增广的判别函数形式表示）：

$$g_i^k(y) = \alpha_i^{(k)T} y$$

其中，$k = 1,2,\dots,l_i$， $i = 1,2,\dots,c$

- 假设ω_i，$i = 1,2,\dots,c$， ω_i类中有l_i个子类，每一类均存在一定数量的训练样本
- （1）初始化：任意给定各类各子类的权值$\alpha_i^{(k)}(0)$，通常可以使用小的随机数；

分段线性判别函数的设计

- （2）在时刻t：当前权值为$\alpha_i^{(k)}(t)$，考虑某个训练样本$y_v \in \omega_j$，找出ω_j类的子类中判别函数最大的子类

$$\alpha_j^{(m)}(t)^T y_v = \max_{k=1,2,\ldots,l_i} \alpha_j^{(k)}(t)^T y_v$$

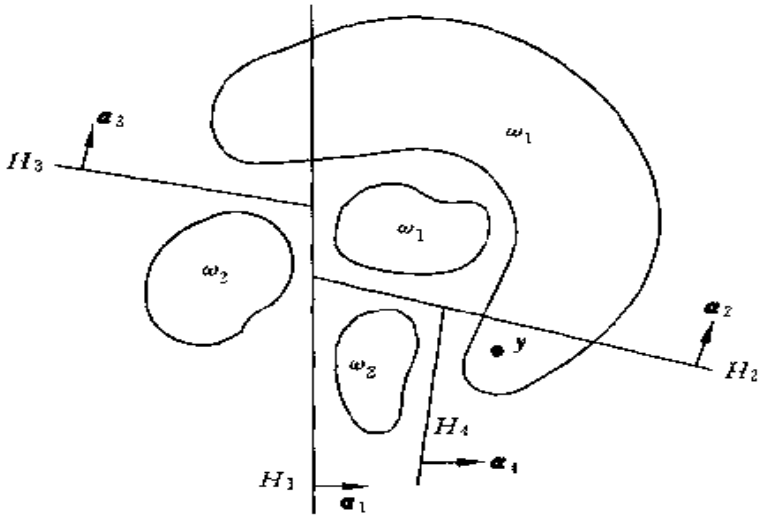
- 考察当前权值对样本y_v的分类情况：
  - 若$\alpha_j^{(m)}(t)^T y_v > \alpha_i^{(k)}(t)^T y_v$， $\forall i = 1, 2, \ldots, c$，$i \neq j$， $k = 1, 2, \ldots, l_i$，即该样本y_v被正确分类，则$\alpha_i^{(k)}(t)$不变
  - 若对某个$i \neq j$，$k = n$，有$\alpha_j^{(m)}(t)^T y_v \leq \alpha_i^{(n)}(t)^T y_v$，则$y_v$被错分，对其中最大者，记为（$i'$，$n'$）
  - 修正

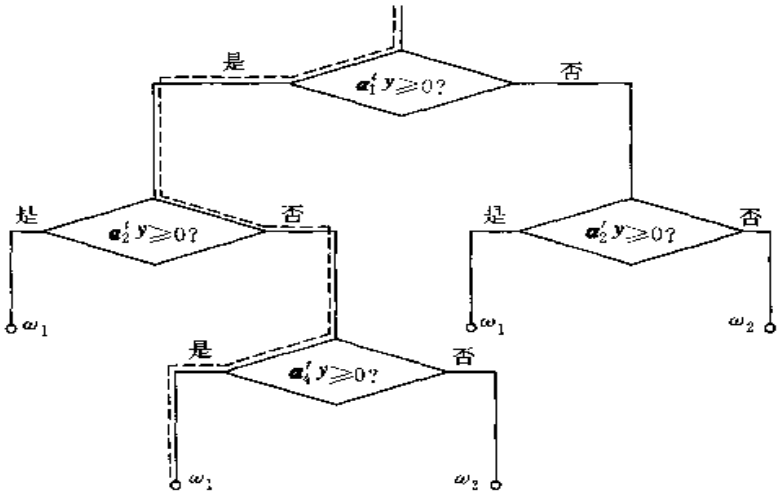
$$\alpha_j^{(m)}(t+1) = \alpha_j^{(m)}(t) + \rho_t y_t$$
$$\alpha_{i'}^{(n')}(t+1) = \alpha_{i'}^{(n')}(t) - \rho_t y_t$$

- （3）对下一个样本重复（2），直到收敛。

分段线性判别函数的设计

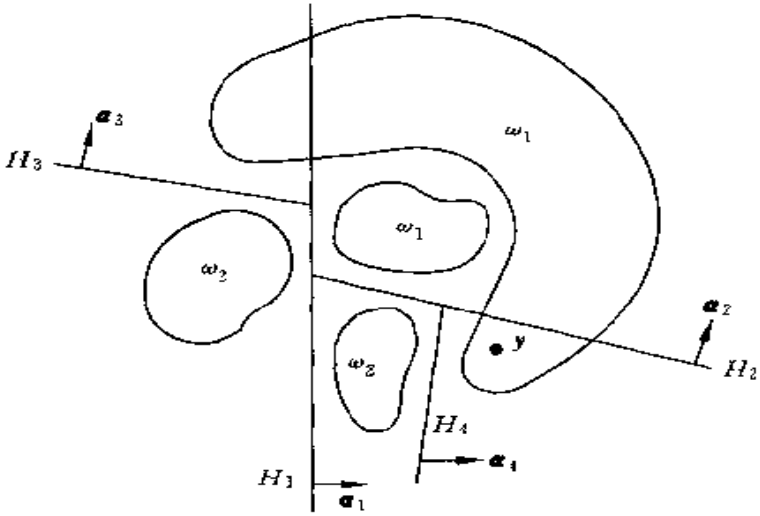
- 第三种情况：未知子类数目
- 可使用**树状分段线性分类器**（二维示例说明）
  - 首先设计一个线性分类器，分成两个子类
  - 若子类中有错分，则在其中再分，直到全部正确分类

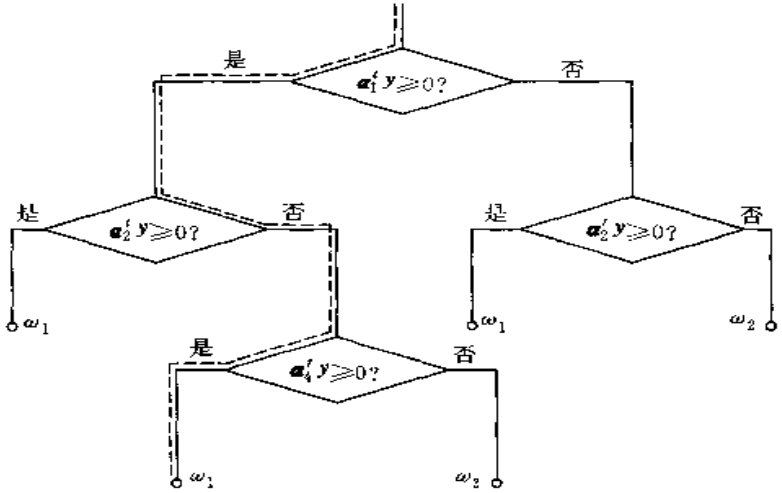




分段线性判别函数的设计

- 第三种情况：未知子类数目
- 对于初始值敏感；不同的权向量搜索算法会导致结果不同。
- 通常：选择分属两类的欧氏距离最小的一对样本，取其垂直平分面的法向量作为 α_1 的初始值，然后求得局部最优解 α_1^* 。





二次判别函数

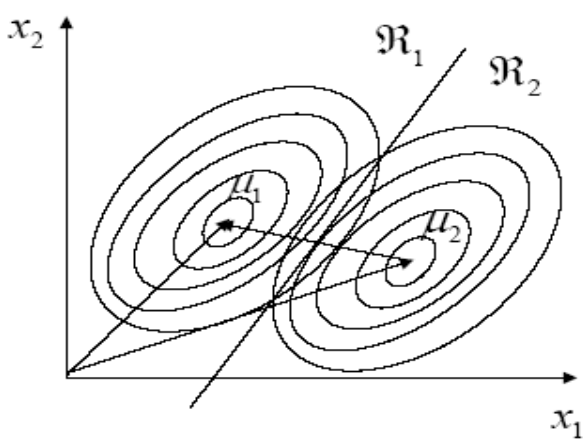
多元正态 - 判别函数和决策面

- 每类协方差矩阵相等，类内各特征相互独立。如果各类先验概率相等，则：

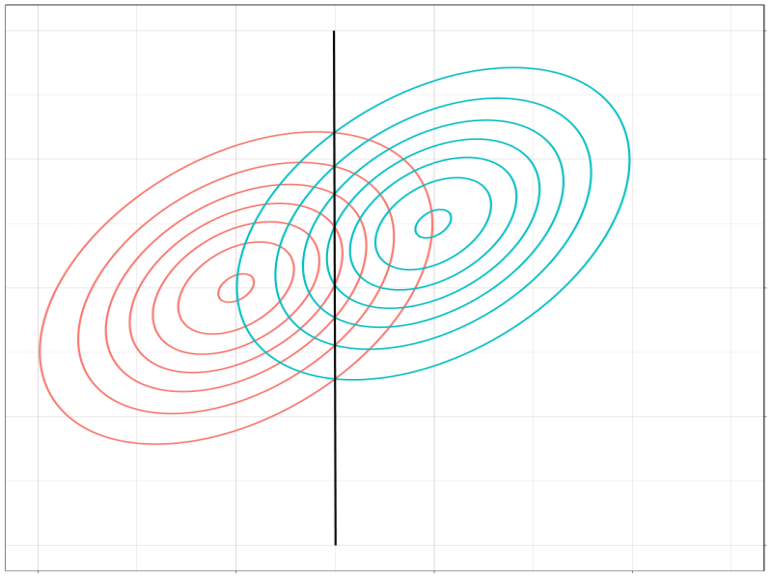
$$x_0 = \frac{1}{2}(\mu_i + \mu_j)$$

此时， x_0为$\mu_i + \mu_j$连线的中点，决策面通过该点。

- 当$P(\omega_i), i = 1, 2, \ldots, c$ 不相等，决策面向先验概率小的方向偏移。

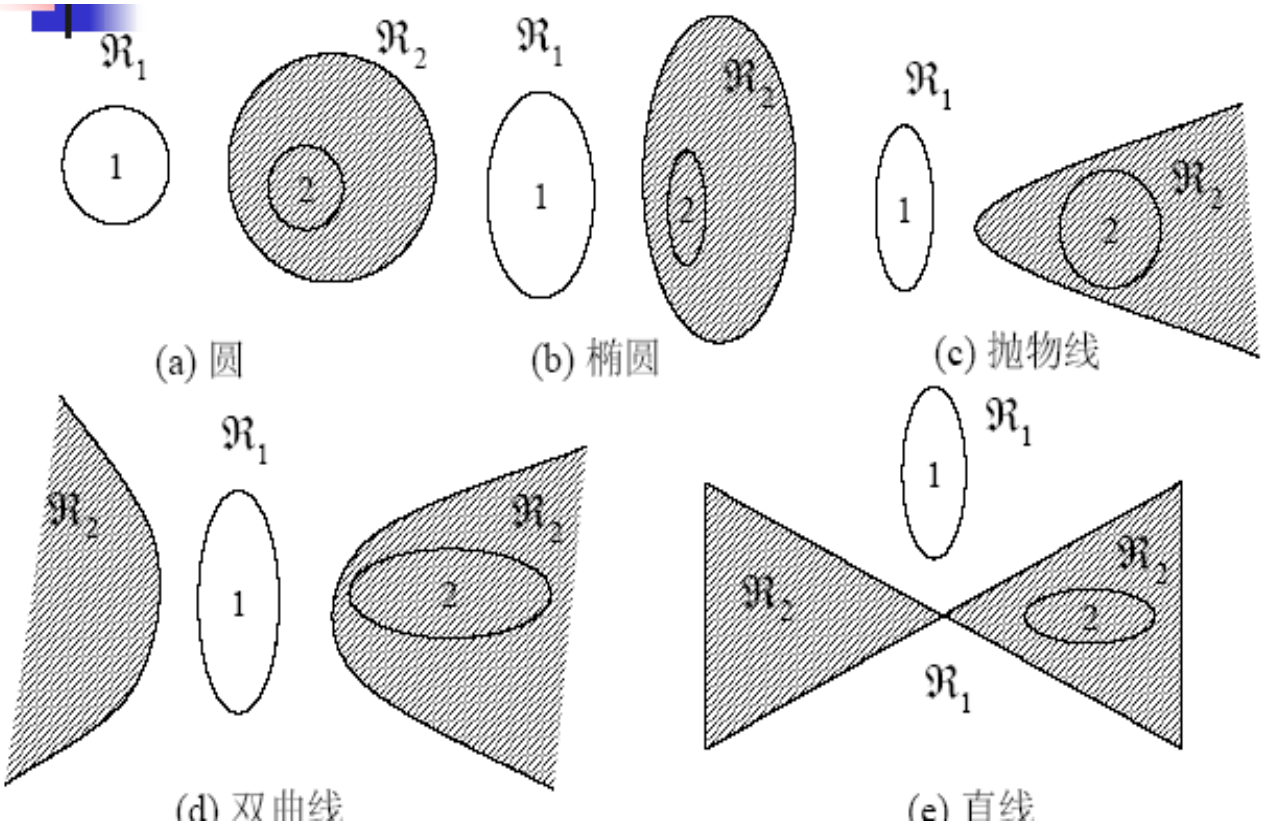


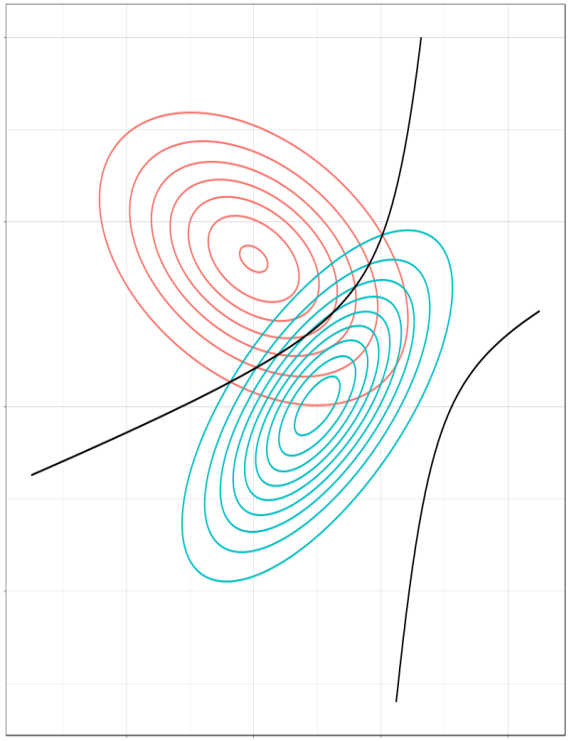
正态分布且$P(\omega_1) = P(\omega_2), \Sigma_1 = \Sigma_2$ 时的决策面



多元正态 - 判别函数和决策面

- 如果假定各先验概率相等，决策面的不同是由于方差项的差异引起的。





二次判别函数

- 在一般正态分布情况下，贝叶斯决策面是二次函数。
- 二次判别函数一般可表示成：

$$g(x) = x^T \overline{W} x + w^T x + w_0$$

$$= \sum_{i=1}^{n} w_{ii} x_i^2 + 2 \sum_{j=1}^{n-1} \sum_{i=j+1}^{\mathrm{n}} w_{ji} x_j x_i + \sum_{j=1}^{n} w_j x_j + w_0$$

其中，$\overline{\mathrm{W}}$是$n \times n$维的实对称矩阵，w为n维向量。

- $g(x)$的系数一共有$l = \frac{1}{2} n(n+3) + 1$，计算起来非常复杂，计算量很大。当样本数量不足时，结果的可靠性和泛化性难以保证。

二次判别函数

- 二次决策面为超二次曲面（超球面，超双曲面等）

（1）在实际中，往往**基于一定的假定**进行求解。若已知样本ω_1分布比较集中，形成单峰，ω_2分布分散，如下图。此时定义ω_1判别函数：

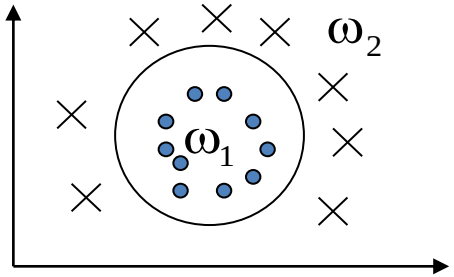
$$g(x) = \boldsymbol{K}^2 - \boxed{(x-\bar{\mu}_1)^T \Sigma_1^{-1}(\boldsymbol{x}-\bar{\boldsymbol{\mu}}_1)},$$

马氏距离的平方

其中：$\bar{\mu}_1$为ω_1类均值，Σ_i为ω_1类协方差；$\boldsymbol{K}^2$为阈值项，它受协方差矩阵和先验概率的影响，决定超平面的大小。

判别规则：$\begin{cases} g(x) > 0, x \in \omega_1 \\ g(x) < 0, x \in \omega_2 \end{cases}$

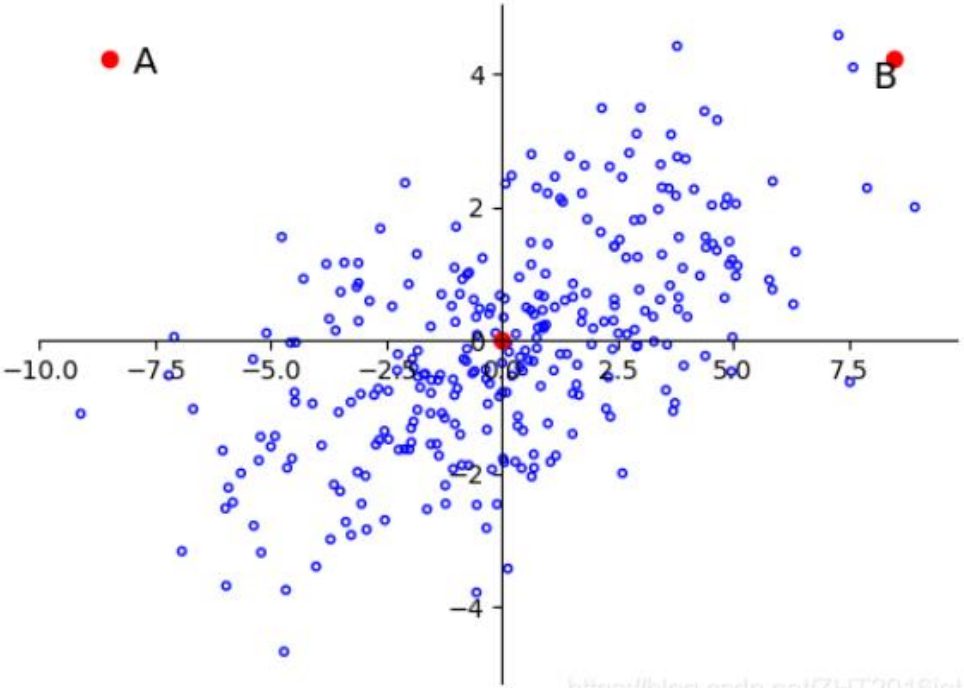
判别平面：$\boldsymbol{g_1(x) = 0}$是个超平面。



ω_1类比较集中

马氏距离

- 马氏距离与欧式距离的比较
  - 欧式距离只取决于两个样本点，与全局统计性质无关；马氏距离考虑方差的特性。
  - 欧氏距离受特征的量纲影响，马氏距离进行了归一化，不会受此影响。



https://blog.csdn.net/ZHT2016iot

二次判别函数

（2）如果ω_1，ω_2都比较集中，那么定义两个判别函数：

$$g_i(x) = \boldsymbol{K}_i{}^2 - (x - \bar{\mu}_i)^T \sum_i^{-1} (x - \bar{\mu}_i)，i = 1,2$$

其中：$\bar{\mu}_i$为ω_1，ω_2均值，$\sum_i$为ω_1，ω_2协方差

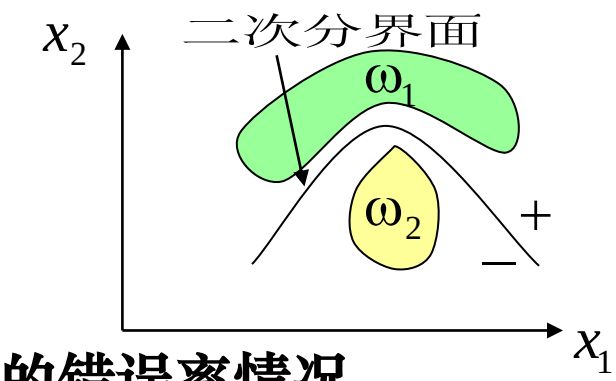
则判别平面方程为：

$$g(x) = g_1(x) - g_2(x) = 0，即：$$

$$-\boldsymbol{x}^T\left(\sum_{\mathbf{1}}^{-\mathbf{1}} - \sum_{\mathbf{2}}^{-\mathbf{1}}\right)\boldsymbol{x} + \mathbf{2}\left(\bar{\boldsymbol{\mu}}_{\mathbf{1}}{}^T\sum_{\mathbf{1}}^{-\mathbf{1}} - \bar{\boldsymbol{\mu}}_{\mathbf{2}}{}^T\sum_{\mathbf{2}}^{-\mathbf{1}}\right)\boldsymbol{x} - \left(\bar{\boldsymbol{\mu}}_{\mathbf{1}}{}^T\sum_{\mathbf{1}}^{-\mathbf{1}}\bar{\boldsymbol{\mu}}_{\mathbf{1}} - \bar{\boldsymbol{\mu}}_{\mathbf{2}}{}^T\sum_{\mathbf{2}}^{-\mathbf{1}}\bar{\boldsymbol{\mu}}_{\mathbf{2}}\right)$$
$$+\left(\boldsymbol{K}_{\mathbf{1}}{}^{\mathbf{2}} - \boldsymbol{K}_{\mathbf{2}}{}^{\mathbf{2}}\right) = \mathbf{0}$$

判别规则为：

$$g(x)\begin{cases} > 0，x \in \omega_1 \\ < 0，x \in \omega_2 \end{cases}$$



可以通过调整两类的阈值$k_1{}^2$,$k_2{}^2$来调整两类的错误率情况。

多层感知器神经网络

人工神经网络发展概述

- 萌芽期（20世纪40年代）

  - 40年代初，美国McCulloch和Pitts从信息处理的角度，研究神经细胞行为的数学模型表达。提出了二值神经元模型。**MP模型**的提出开始了对神经网络的研究进程。

W. S. McCulloch
(1898-1969)

W. Pitts
(1923-1969)

  - 1949年，心理学家Hebb提出著名的Hebb学习规则，即由神经元之间结合强度的改变来实现神经学习的方法。

Donald O. Hebb
(1904-1985)

人工神经网络发展概述

- 第一次高潮期（20世纪50-60年代）
  - 现代串行计算机的奠基人Von Neumann注意到计算机与人脑的差异，对类似与神经网络的分布系统做了许多研究
  - 50年代末期，Rosenblatt提出感知机模型，它基本符合神经生理学的原理。在60代掀起了神经网络研究的第一次高潮。

Von Neumann
(1903-1957)

Frank Rosenblatt
(1928-1971)

人工神经网络发展概述

- 反思期（1969-1980年）

  - 1969年，人工智能创始人之一的Minsky和Papert以出版了《感知器》，从数学上深入分析了感知器的原理，指出其局限性。

  - Minsky的结论是悲观的，神经网络的研究进入了低潮。主要有自适应共振理论，自组织映射，认知机网络模型理论，BSB模型等等，为神经网络的发展奠定了理论基础。

Marvin Lee Minsky
(1921-)

Seymour Papert
(1928-)

人工神经网络发展概述

- 第二高潮期(1981~1990)

  - 1982年，Hopfield提出了人工神经网络的一种数学模型，引入了能量函数的概念，设计出用电子线路实现这一网络的方案，大大促进了人工神经网络的研究。
  - 1986年，Rumelhart及Hinton等学者提出了多层感知器的反向传播算法，解决了多层前向神经网络的学习问题，证明了多层神经网络具有很强的学习能力。

John J. Hopfield
(1933-)

David Rumelhart
(1942-2011)

Geoffrey Hinton
(1947-)

人工神经网络发展概述

●再认识与应用研究期(1991~2006年)

问题：

- 应用面还不够宽
- 结果不够精确

研究：

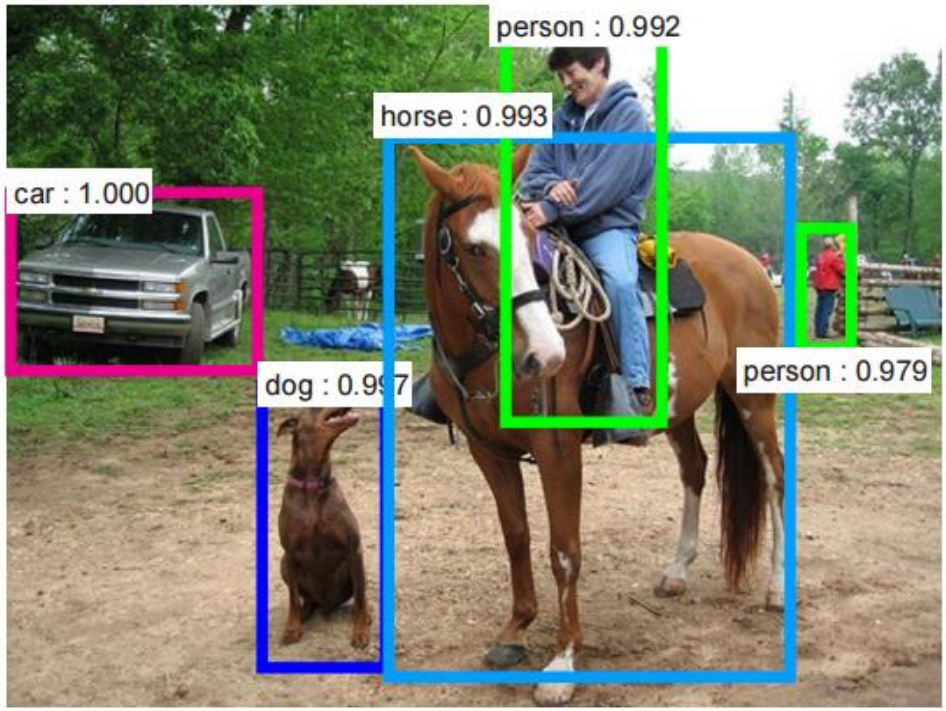
- 开发现有模型的应用，并在应用中根据实际运行情况对模型、算法加以改进，以提高网络的训练速度和运行的准确度。
- 在理论上寻找新的突破，建立新的专用/通用模型和算法。
- 进一步对生物神经系统进行研究，不断地丰富对人脑的认识。

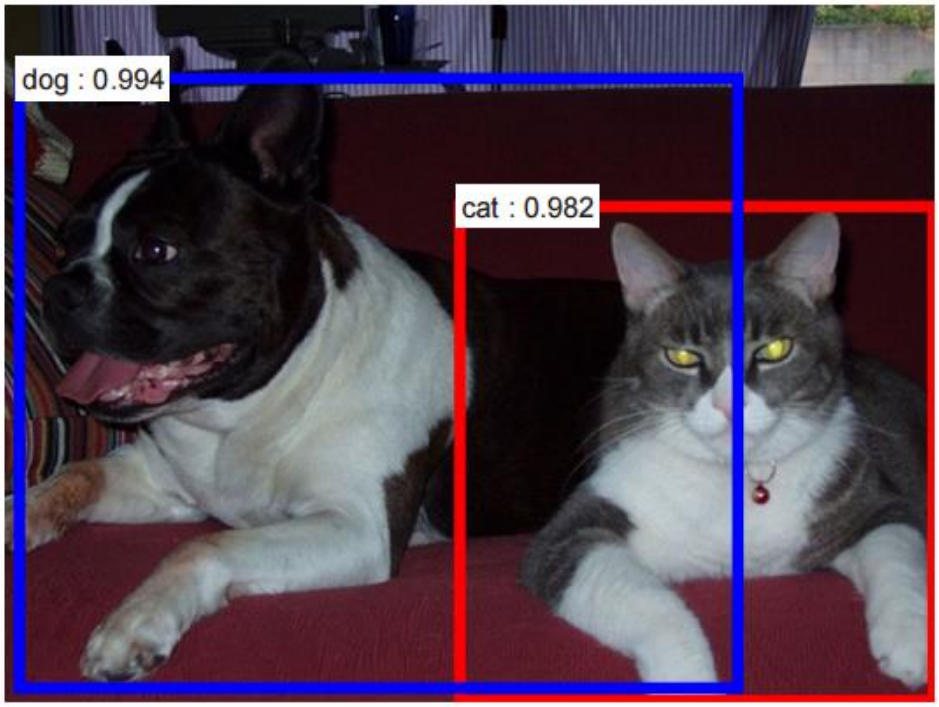
人工神经网络发展概述

●深度神经网络时期(2006~)

- 2006年，Hinton提出了更有效的训练神经元的新方法，通过“逐层初始化（layer-wise pre-training）”来克服多层神经网络在训练上的难度。
- 2012年，Alex Krizhevsky 在ImageNet图像分类竞赛中取得了里程碑式的突破，Top5分类准确率由74%提升至85%.

人工神经网络发展概述


person : 0.992
horse : 0.993
car : 1.000
dog : 0.997
person : 0.979


dog : 0.994
cat : 0.982

人工神经网络发展概述

35

55

75

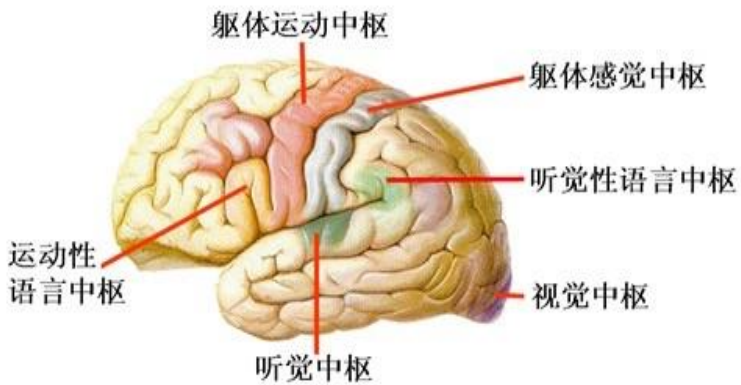
27

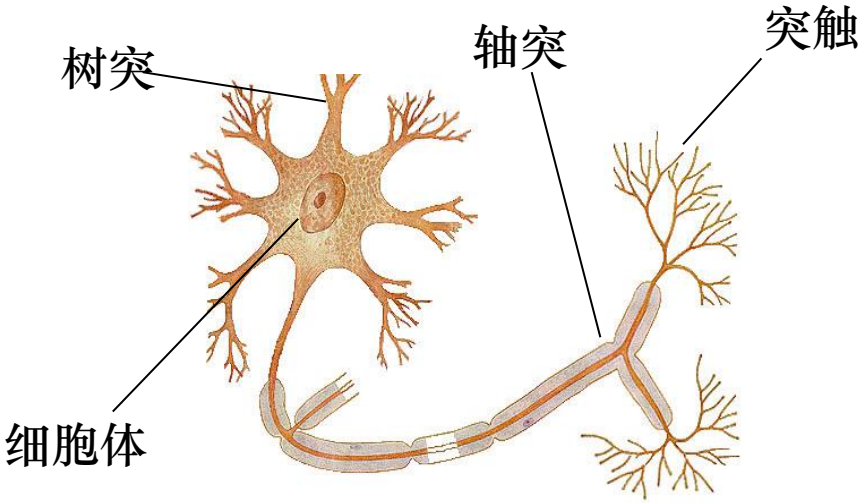
47

67

A
B
C
D

神经元结构





- 人类的大脑皮层大约有14,000,000,000个神经细胞，亦称为神经元。每个神经元有数以千计的通道同其它神经元广泛相互连接，形成复杂的生物神经网络。
- 生物神经细胞功能比较简单，需要通过很多神经元一起协作完成复杂功能，通过数以万计的神经细胞以一定的连接方式或信息传递方式进行协作，这些神经元可以看做构成一个网络，就是神经网络。

神经元结构

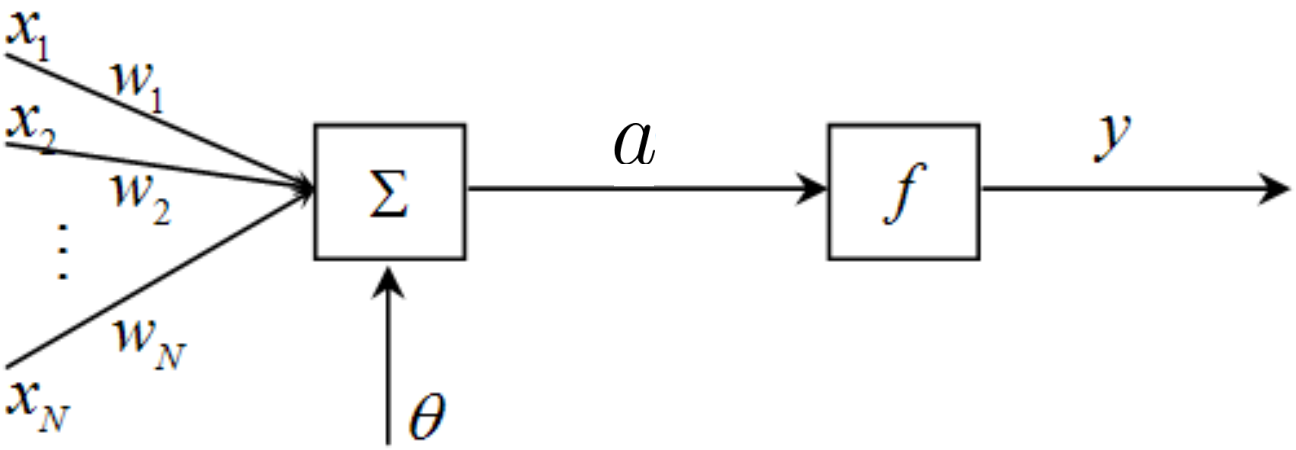
- **工作机制**
  - 一个神经元有两种状态：**兴奋和抑制；**
  - 平时处于抑制状态的神经元，其树突和胞体接收其他神经元由突触传来的兴奋电位，多个输入在神经元中以代数和的方式**叠加**；
  - 如果输入兴奋电位总量超过某个**阈值**，神经元会被**激发**进入兴奋状态，发出输出脉冲，并由突触传递给其他神经元。
  - 神经元被触发后进入不应期，在不应期不能被触发，然后阈值逐渐下降，恢复兴奋性。

神经元结构

- **六个基本特征**
  - 1）神经元及其**联接**；
  - 2）神经元之间的**联接强度**决定信号传递的强弱；
  - 3）神经元之间的联接强度是**可以随训练改变**的；
  - 4）信号可以是起**刺激**作用的，也可以是起**抑制**作用的；
  - 5）一个神经元接受的信号的**累积效果**决定该神经元的状态；
  - 6) 每个神经元可以有一个“**阈值**”。

人工神经元的MP模型

- **MP模型**
  - 是一种人工神经元的数学模型，它最早是由美国的McCulloch和Pitts提出的神经元模型，是大多数神经网络模型的基础。



人工神经元的MP模型

- 人工神经元是仿照生物神经元提出的，神经元可以有N个**输入**：

$$x_1, x_2, \ldots, x_N$$

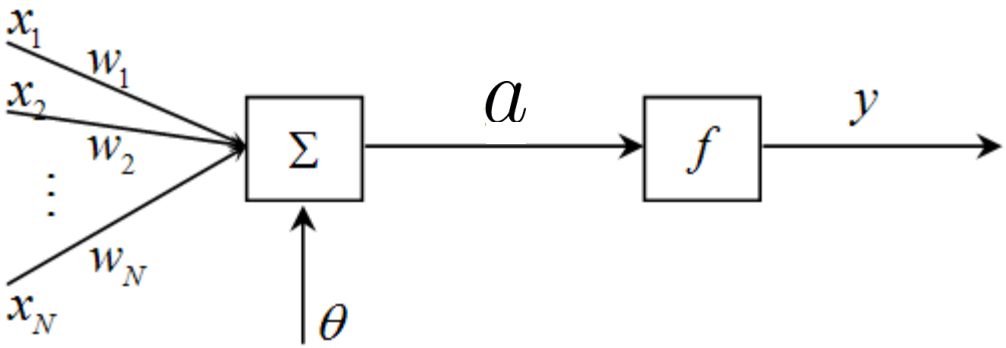
- 每个输入端与神经元之间有一定的**联接权值**：

$$w_1, w_2, \ldots, w_N$$

- 神经元总的输入为对每个输入的加权求和，同时减去**阈值**θ。a代表神经元的活跃值，即**神经元状态**：

$$a = \sum_{i=1}^{N} w_i x_i - \theta = \sum_{i=0}^{N} w_i x_i$$

人工神经元的MP模型



- 神经元的输出y是对a的映射：

$$y = f(a) = f\left(\sum_{i=0}^{N} w_i x_i\right)$$

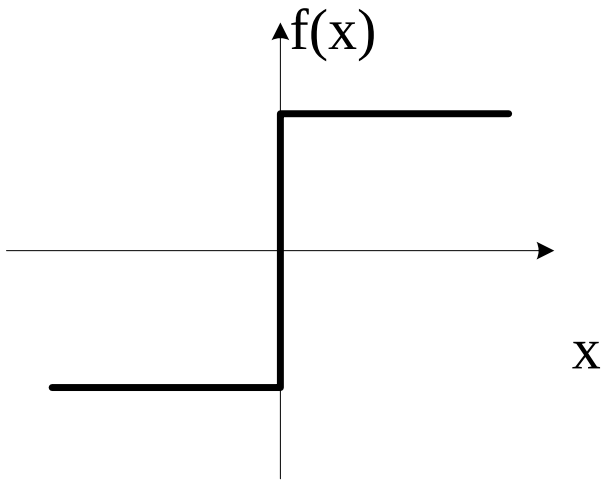
***f* 称为激活函数**(激励函数，输出函数)，有很多形式，例如阶跃函数、符号函数等。

人工神经元的MP模型

- MP模型采用**阶跃函数**作为激活函数：

$$f(x)=\begin{cases}1, & x\geq 0\\ -1, & x<0\end{cases}$$

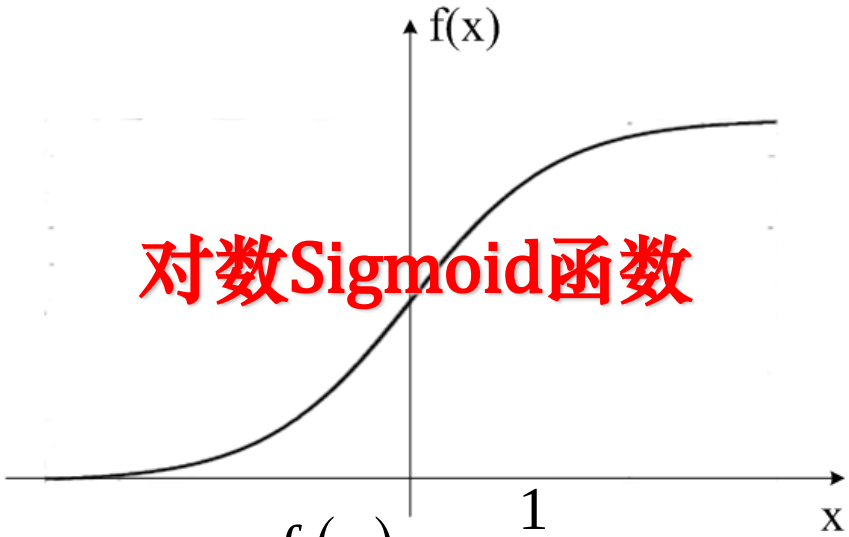
$$y=\operatorname{sign}\left(\sum_{i=0}^{N} w_i x_i\right)$$



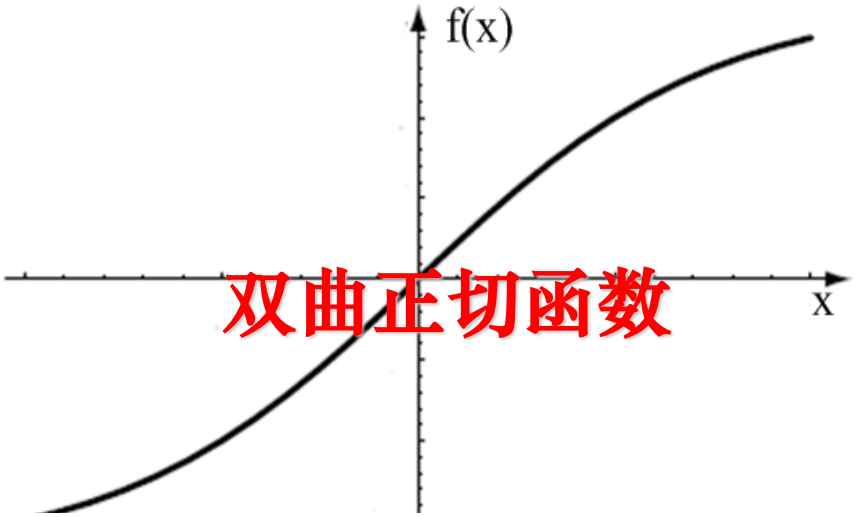
- 当把可能的输出值看做两类，神经元就可以看作是一个线性分类器。
- 从几何上看，感知器神经元用超平面把特征分成了两个区域。
- MP模型高度抽象，但反映了自然神经元的关键特性。

人工神经元的MP模型

- **激活函数：**
  - 线性函数
  - 非线性斜面函数(Ramp Function)
  - Sigmoid、tanh

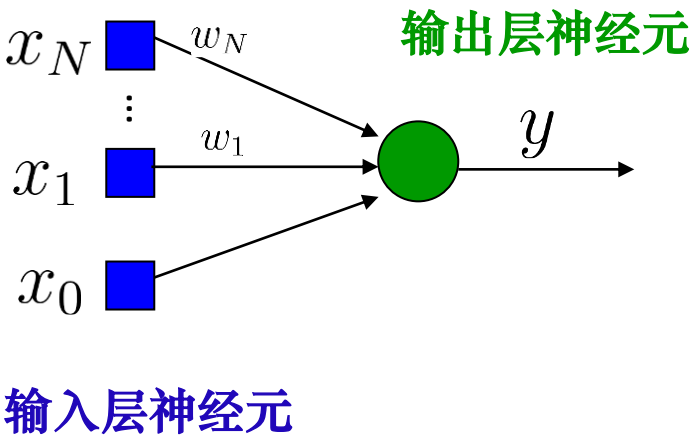


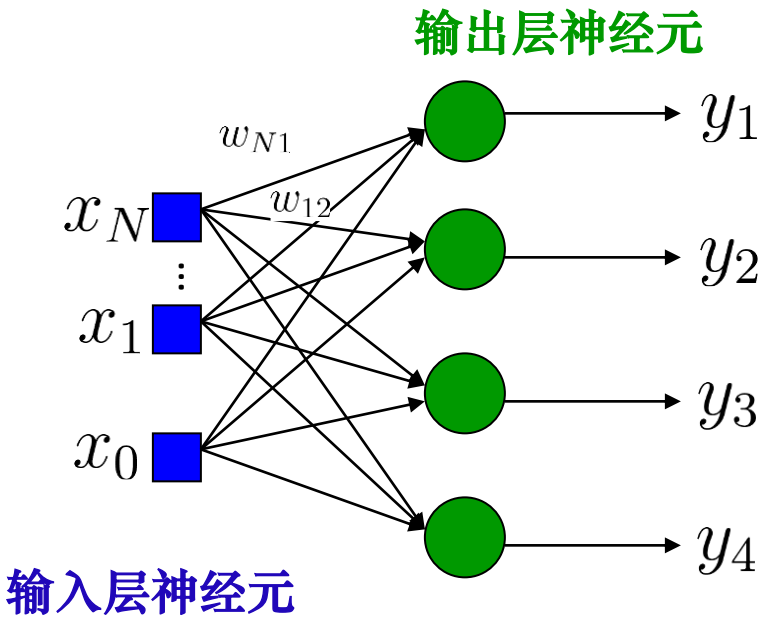
$$f(x)=\frac{1}{1+e^{-x}}$$



$$f(x)=\tanh(x)=\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}=\frac{2}{1+e^{-2x}}-1$$

感知器





$$y = f(a) = f\left(\sum_{i=1}^{N} w_i x_i\right)$$

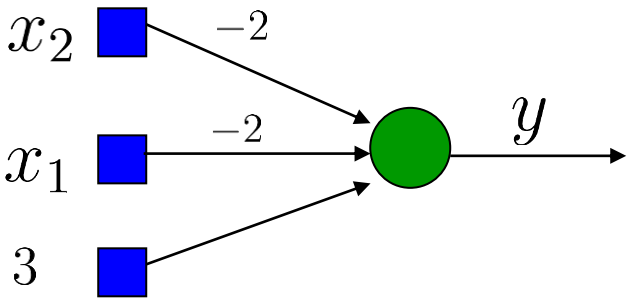
$$y_j = f(a_j) = f\left(\sum_{i=1}^{N} w_{ij} x_i\right)$$

感知器

- 示例：与非门

真值表

		x_1	
		0	1
x_2	0	1	1
	1	1	0



x_1	x_2	a	y
1	1	-1	0
1	0	1	1
0	1	1	1
0	0	3	1

感知器

- **线性不可分**问题：感知器模型的局限

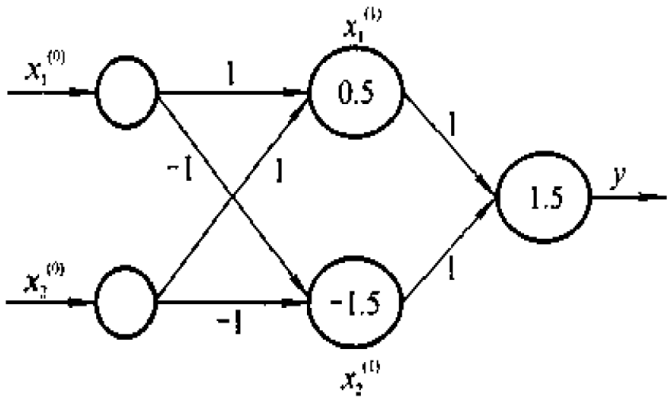
例：异或(XOR)问题，即： $a \oplus b = (\neg a \wedge b) \vee (a \wedge \neg b)$

		x_1	
		0	1
x_2	0	0	1
	1	1	0

解决方法：三层感知器。通过将多个神经元分层组合起来，实现复杂的空间形状的分割。

多层感知器

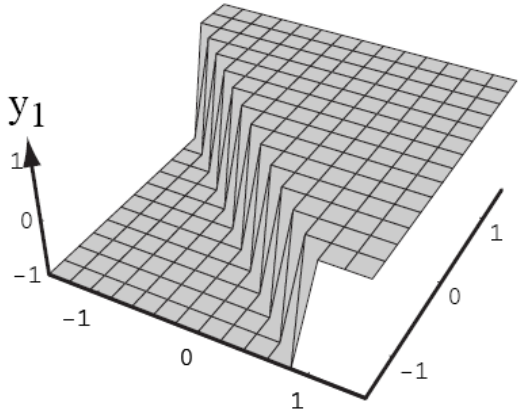
- 对于异或问题，用一个简单的三层感知器就可得到解决。



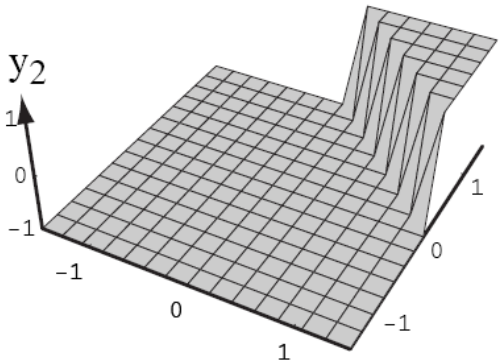
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

理论证明，三层感知器可以实现任意的逻辑运算，在激活函数为Sigmoid函数的情况下，可以逼近任何非线性多元函数。

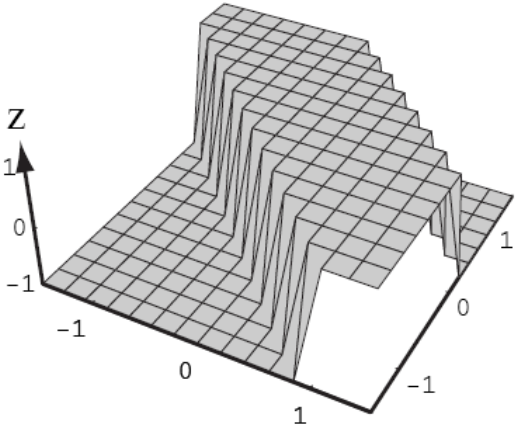
多层感知器



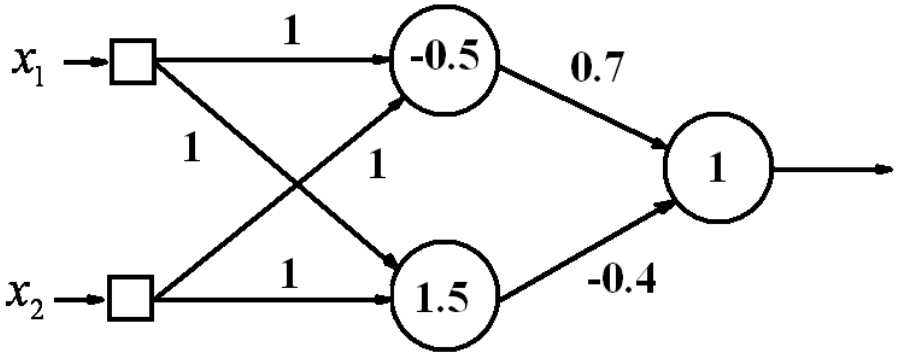
$x_1+x_2+0.5=0$



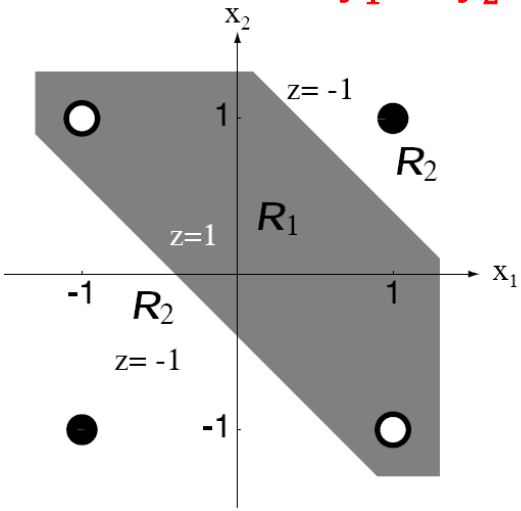
$x_1+x_2-1.5=0$



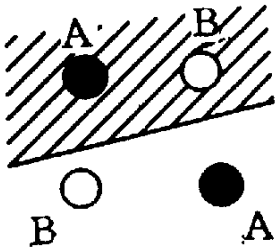
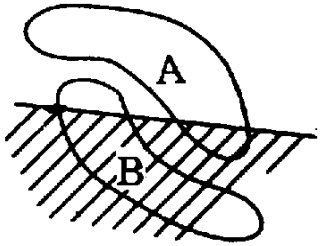
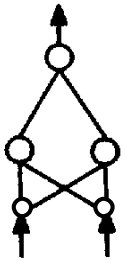
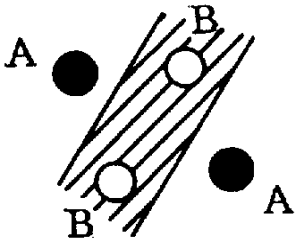
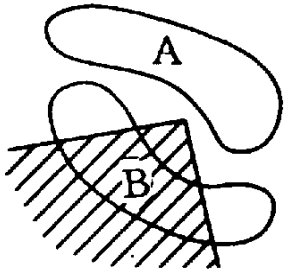
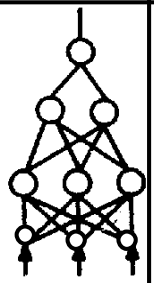
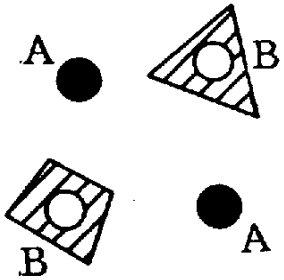
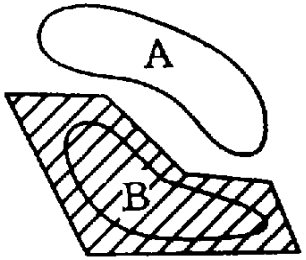
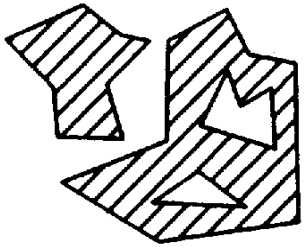
$0.7*y_1-0.4y_2-1=0$



x_1 , x_2 = 1 or -1



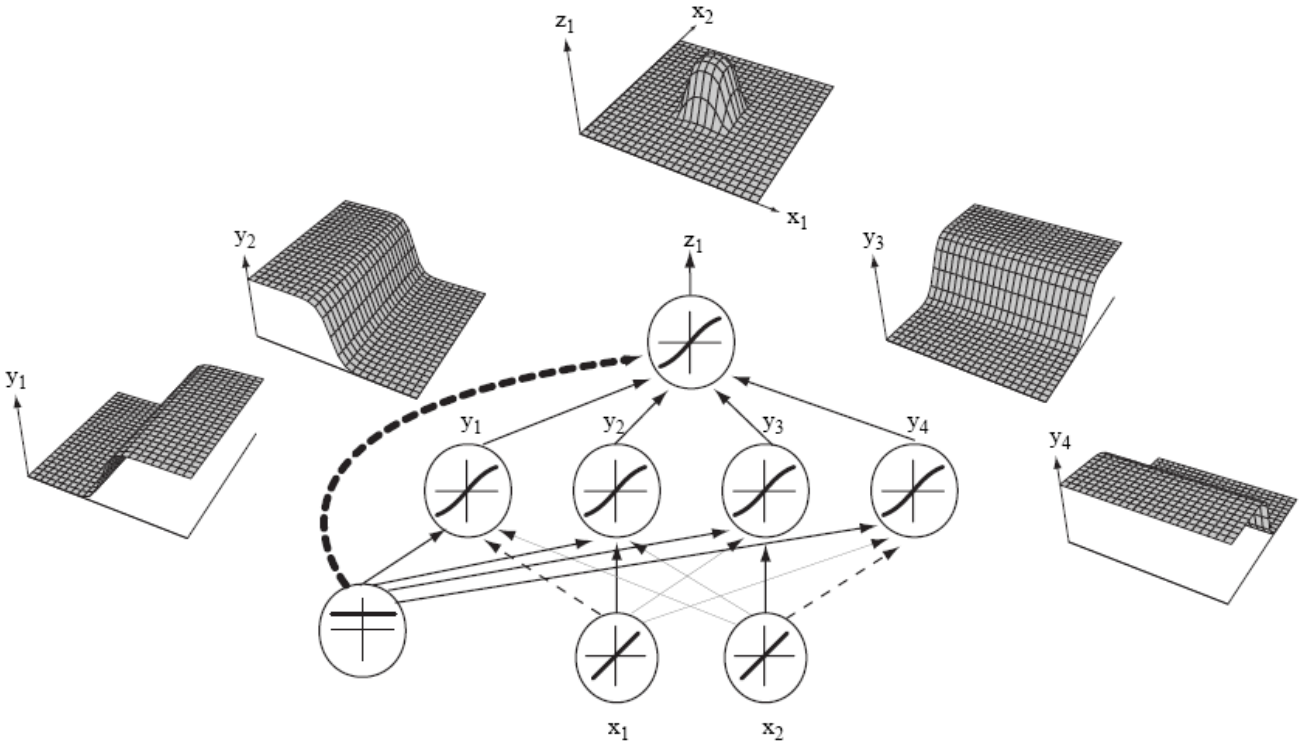
感知器小结

感知器结构	异或问题	复杂问题	判决域形状	判决域
无隐层				半平面
单隐层 				凸　域
双隐层 				任意复杂形状域

感知器小结

- **输出函数：S型函数**

若给出足够多隐单元，任何从输入到输出的连续函数都可用三层神经网络以任意精度近似。



感知器小结

- 感知器的学习过程与求线性判决函数(单样本感知器算法)的过程是等价的；
- 两层感知器(一个输入层一个输出层)只能用于解决线性可分的问题；
- 只要隐层和隐层单元数足够多，多层感知器网络可实现任何模式分类；
- 但是，多层网络的权值如何确定，即网络如何进行学习，在感知器上没有得到解决。

反向传播算法

Sigmoid函数

- **Sigmoid替代感知器中的阈值函数（阶跃函数）来构造神经元**

$$\sigma(a) = \frac{1}{1+\exp(-a)} \qquad a = w^T x$$

Sigmoid函数性质：

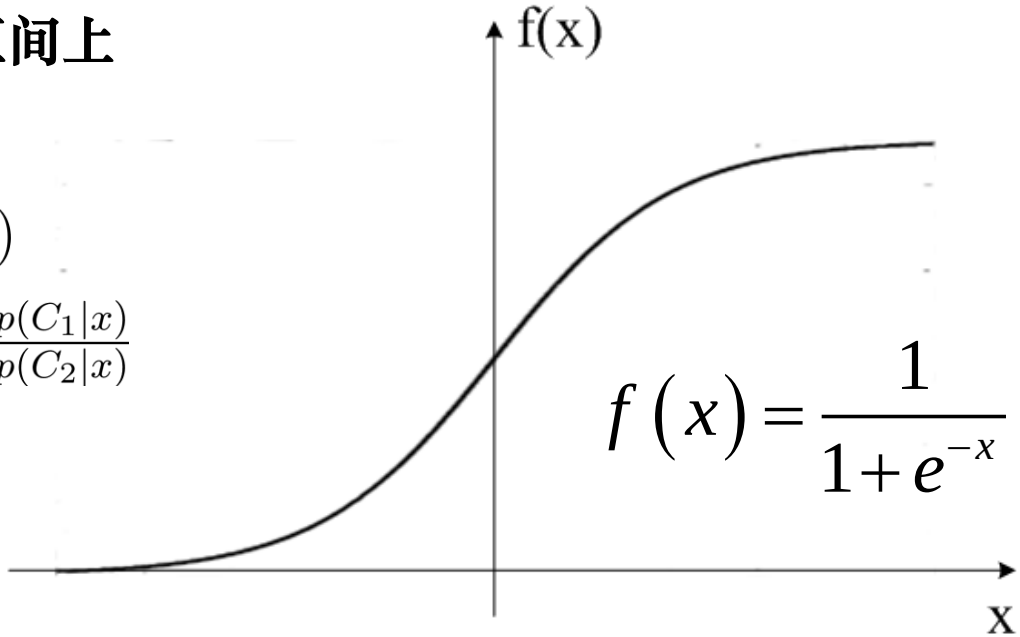
①可以将连续值映射到0和1区间上

②可导性 $\frac{d\sigma}{da} = \sigma(1-\sigma)$

③对称性 $\sigma(-a) = 1 - \sigma(a)$

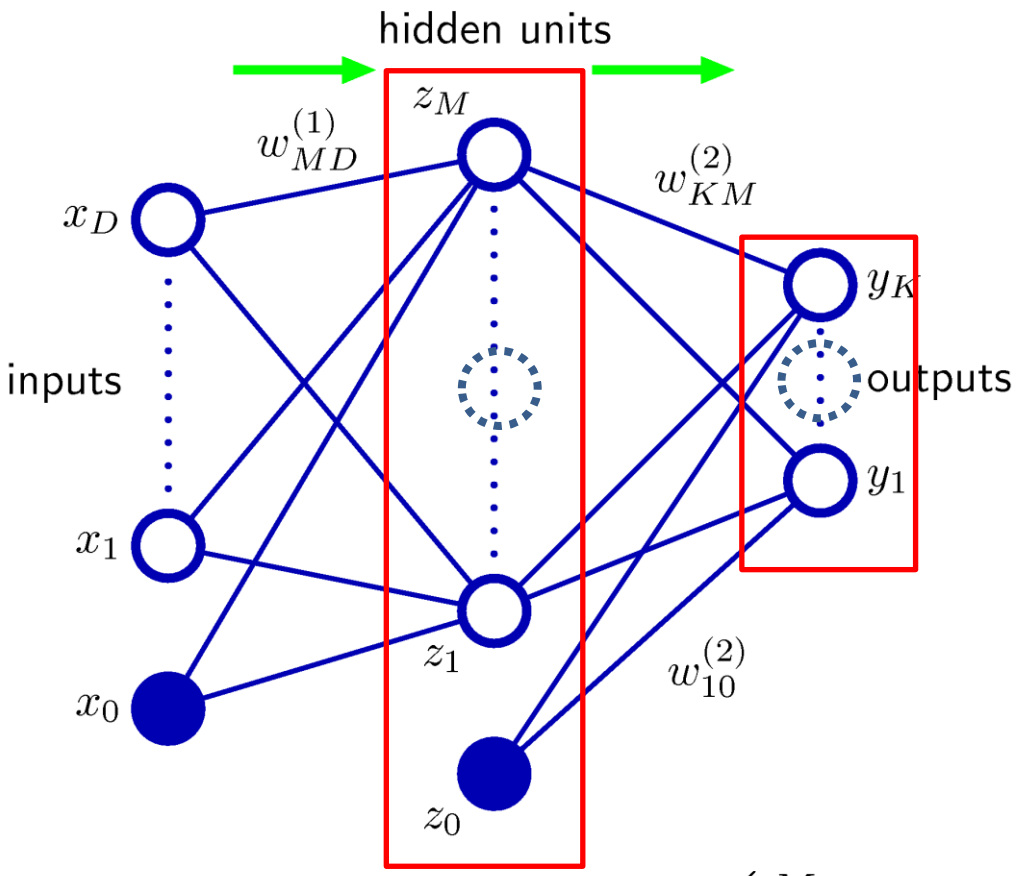
④反转性 $a = ln(\frac{\sigma}{1-\sigma}) = ln\frac{p(C_1|x)}{p(C_2|x)}$

对数Sigmoid函数



$$f(x) = \frac{1}{1+e^{-x}}$$

前馈神经网络



$$a_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i$$

$$z_j = h(a_j)$$

$$a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j$$

$$y_k = \sigma(a_k)$$

$$\sigma(a) = \frac{1}{1+\exp(-a)}$$

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma\left(\sum_{i=0}^{M} w_{kj}^{(2)} h\left(\sum_{i=0}^{D} w_{ji}^{(1)} x_i\right)\right)$$

BP(Back-Propagation)算法

- BP神经网络模型拓扑结构包括输入层(input layer)隐层(hidden layer)和输出层(output layer)。能学习和存贮大量的输入-输出模式映射关系，而无需事前揭示描述这种映射关系的数学方程。
- BP 神经网络是一种按误差逆传播算法训练的多层前馈网络，是目前应用最广泛的神经网络模型之一。
- 基本BP算法包括两个方面：信号的前向传播和误差的反向传播。即计算实际输出时按从输入到输出的方向进行，而权值和阈值的修正从输出到输入的方向进行，通过反向传播来不断调整网络的权值和阈值，使网络的误差平方和最小。利用输出后的误差来估计输出层的直接前一层的误差，再用这个误差估计更前一层的误差，如此一层一层的反传下去，就获得了所有其他各层的误差估计。

BP反向传播算法

- 确定网络结构，进行权值初始化。设定准则函数E，

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2$$

- 寻找一个w，使得 E(w) 最小。

$$\nabla E(\mathbf{w}) = 0$$

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$$

BP反向传播算法

- **样本集：** $\{\mathrm{x}_n, \mathbf{t}_n\}_{n=1}^{N}, \mathrm{x} \in \mathbb{R}^D, \mathbf{t} \in \mathbb{R}^3$

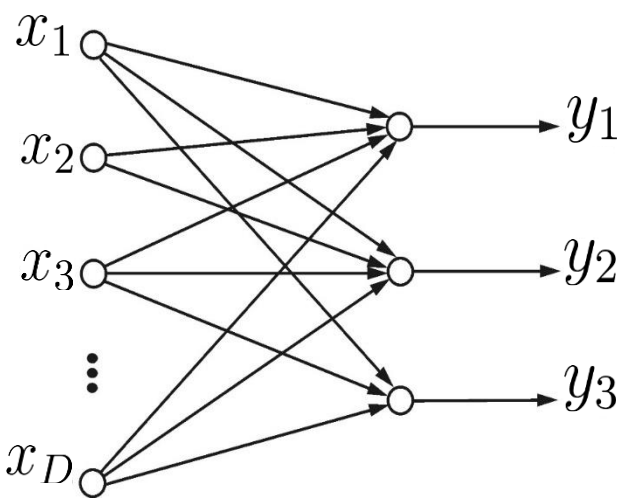
- **选取输入样本，计算对应的期望输出：**

$$y_k = \sum_{i=1}^{D} w_{ki} x_i$$

- **利用网络期望输出和实际输出，计算误差函数对输出层的各神经元的偏导数。**

$$E_n(\mathrm{w}) = \frac{1}{2} \sum_{k=1}^{3} (y_{nk} - t_{nk})^2$$

$$\Rightarrow \frac{\partial E_n}{\partial w_{ki}} = (y_{nk} - t_{nk}) x_{ni}$$



- 示例：线性模型

BP反向传播算法

- **练习：** 对非线性： $y_k = h(a_k) = h(\sum_i w_{ki} x_i)$

计算 $\frac{\partial E_n}{\partial w_{ki}}$ $E_n = \frac{1}{2}\sum_k (y_k - t_k)^2$

$$\frac{\partial E_n}{\partial w_{ki}} = \frac{\partial E_n}{\partial y_k}\frac{\partial y_k}{\partial a_k}\frac{\partial a_k}{\partial w_{ki}}$$

$$= (y_k - t_k)\frac{\partial y_k}{\partial a_k}\frac{\partial a_k}{\partial w_{ki}}$$

$$= (y_k - t_k)h'(a_k)\frac{\partial a_k}{\partial w_{ki}}$$

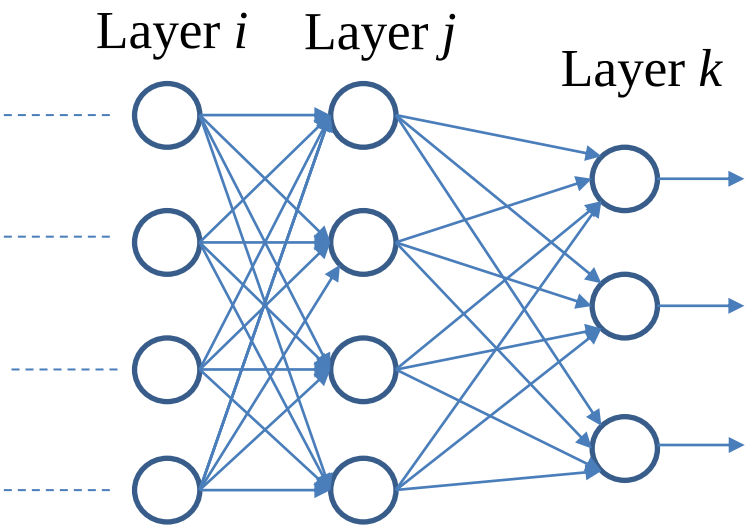
$$= (y_k - t_k)h'(a_k)x_i$$

BP反向传播算法

- 对于包含隐含层的网络，需要在**前馈**的过程中计算隐含层输入和输出：

$$a_j = \sum_i w_{ji} z_i \qquad z_j = h(a_j)$$

- 利用输出层输出、隐含层到输出层的连接权值、隐含层输出，计算误差函数**对隐含层各神经元的偏导数**：



$$\frac{\partial E_n}{\partial w_{ji}} = \boxed{\frac{\partial E_n}{\partial a_j}} \frac{\partial a_j}{\partial w_{ji}}$$

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j} \qquad \frac{\partial a_j}{\partial w_{ji}} = z_i$$

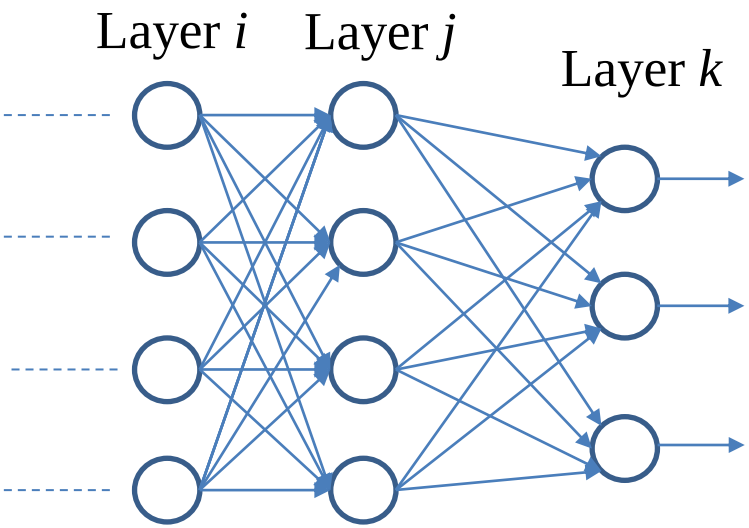
$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

- δ_j标识输出误差反向传播到该层的误差对于权值的导数

BP反向传播算法

- 对于输出层而言：$\delta_k = y_k - t_k$
- 对于隐藏层而言：$\delta_j \equiv \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$



- δ_j标识输出误差反向传播到该层的误差对于权值的导数

BP反向传播算法

- **练习：** $z_j = h(a_j) \quad a_k = \sum_j w_{kj} z_j$

推导 $\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$

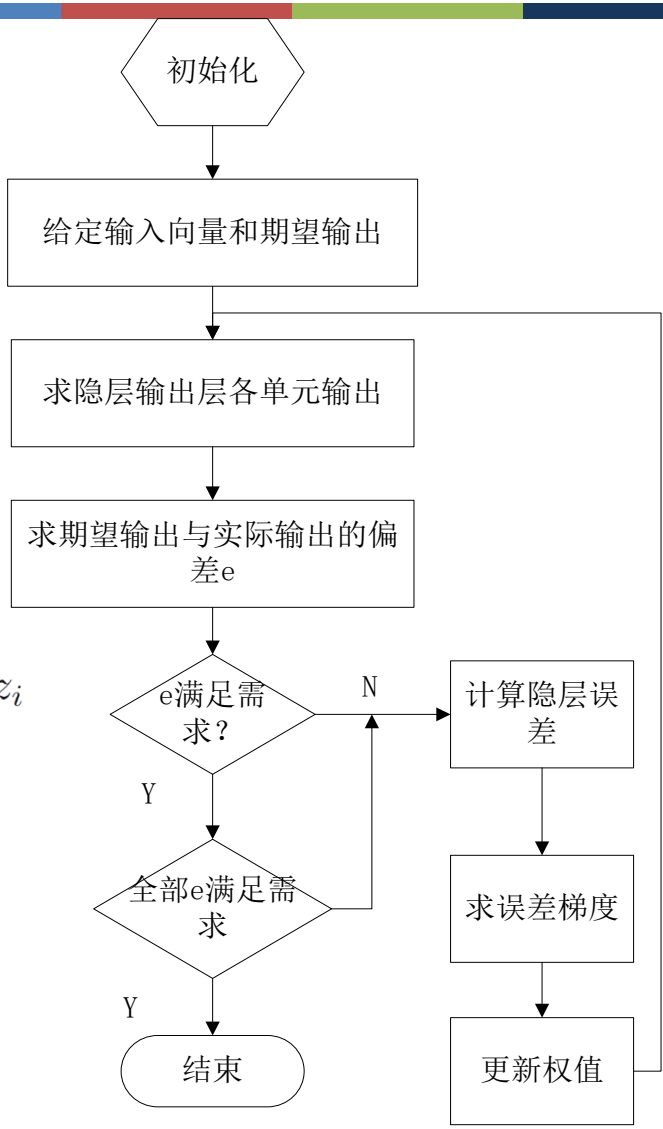
$$\delta_j = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j} = \sum_k \delta_k \frac{\partial a_k}{\partial a_j}$$

$$= \sum_k \delta_k \frac{\partial a_k}{\partial z_j} \frac{\partial z_j}{\partial a_j} = \sum_k \delta_k w_{kj} \frac{\partial h(a_j)}{\partial a_j}$$

$$= h'(a_j) \sum_k \delta_k w_{kj}$$

BP反向传播算法

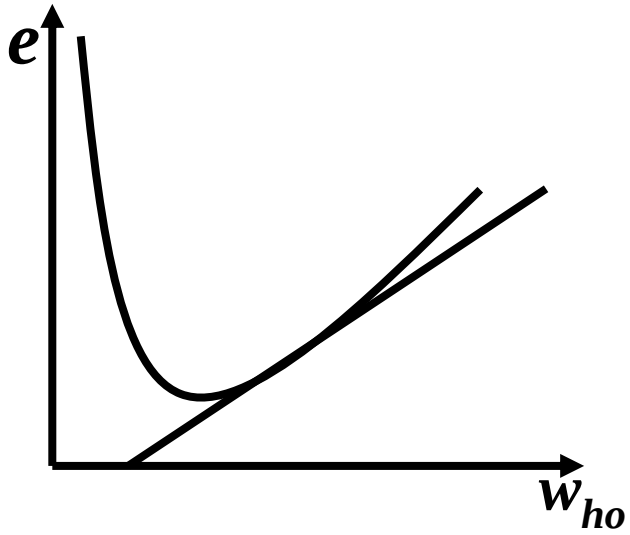
- 初始化权重 w_{ij}；
- 对于输入的训练样本，求取每个节点输出和最终输出层的输出值；
- 对输出层求取 $\delta_k = y_k - t_k$；
- 对于隐藏层求取 $\delta_j = h'(a_j)\sum_k w_{kj}\delta_k$ ；
- 求取输出误差对于每个权重的梯度 $\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$
- 更新权重 $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta\nabla E(\mathbf{w}^{(\tau)})$
- 计算全局误差，判断是否进入下一轮迭代



BP标准算法直观解释

- 情况一、直观表达

当误差对权值的偏导数大于零时，权值调整量为负，实际输出大于期望输出，权值向减少方向调整，使得实际输出与期望输出的差减少。

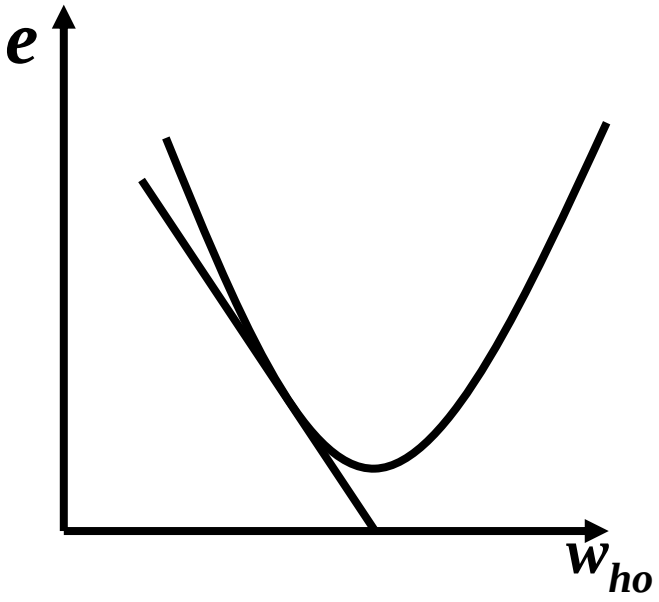


$\frac{\partial e}{\partial w_{ho}}$ >0，此时Δw_{ho}<0

BP标准算法直观解释

- 情况二、直观表达

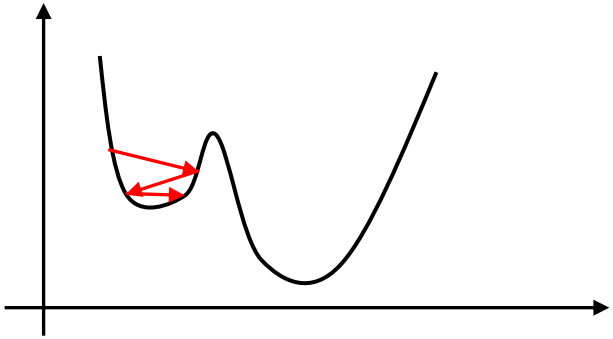
当误差对权值的偏导数小于零时，权值调整量为正，实际输出少于期望输出，权值向增大方向调整，使得实际输出与期望输出的差减少。



$\frac{\partial e}{\partial w_{ho}}$ <0, 此时Δw_{ho}>0

基本BP算法的缺陷

- BP算法因其简单、易行、计算量小、并行性强等优点，是传统神经网络训练采用最多也是最成熟的训练算法之一。
- 其算法的实质是求解误差函数的最小值问题，由于它采用非线性规划中的最速下降方法，按误差函数的负梯度方向修改权值，因而通常存在以下问题：
  - 学习效率低，收敛速度慢
  - 易陷入局部极小状态



  - 后续：变步长；引入“惯性项”

隐含层的层数与节点数的改进

- BP神经网络的构建必须注意隐含层神经元数的选择：
  - 如果隐含层神经元数过少，BP神经网络难以建立复杂的映射关系，网络预测误差较大，
  - 如果隐含层神经元数过多，网络学习的时间增加，并且可能出现过拟合的现象，就是把样本中非规律性的内容（如噪声等）也学会记忆，从而出现训练样本准确，但是其他样本预测误差较大（over-fitting）。

隐含层的层数与节点数的改进

- 如何确定隐含层的神经元个数?
- 目前没有什么成熟的理论能够确定各层神经元的神经元个数和含有几层网络，大多数还是靠经验
- 最佳隐含层神经元数选择可参考如下经验公式:

m为隐含层神经元数，

n为输入层神经元数， l为输出层神经元数

a为1至10之间的常数

$$m = \sqrt{n + l} + a \qquad m = \log_2 n$$

$$m = \sqrt{nl} \qquad \ldots\ldots$$

隐含层的层数与节点数的改进

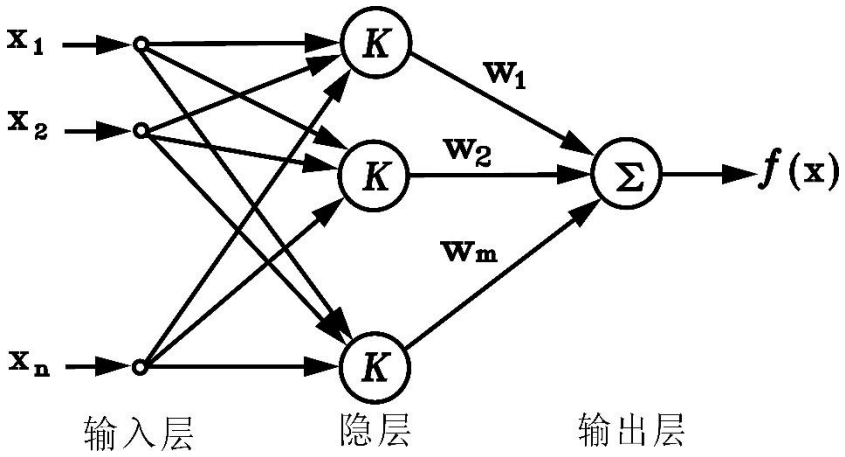
- 实际问题中，隐含层神经元数的选择首先是参考公式来确定神经元的大致范围，然后试凑法确定最佳点数
- 注意对于一般问题BP神经网络的分类误差随着隐含层神经元的增加呈现先减少后增加的趋势

其他常用传统神经网络模型

- 前馈型神经网络：径向基函数网络（RBF）
- 反馈型神经网络：Hopfield网络
- 竞争学习神经网络：自组织特征映射神经网络（SOM）

径向基函数网络（RBF）

- 基本结构：



- 网络特点：只有一个隐层，用一组径向基函数(Radial Basis Function，简称RBF)的加权和来实现某种函数逼近。
- 通常定义为空间中任一点x到某一中心x_c之间欧氏距离的单调函数，可记作$k(\|x - x_c\|)$，其作用往往是局部的，即当x远离x_c时函数取值很小。

径向基函数网络（RBF）

- 最常用的径向基函数是高斯核函数，形式为：

$$k(\|x - x_c\|) = \exp\left\{-\frac{\|x - x_c\|^2}{2\delta^2}\right\}$$

其中x_c为核函数中心，σ为函数的宽度参数，控制了函数的径向作用范围。

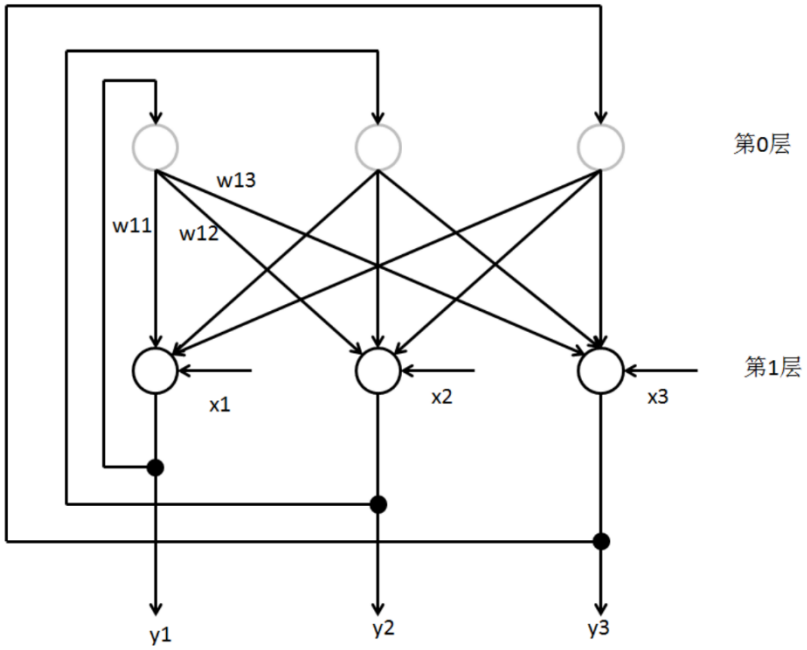
- 在典型的RBF网络中有3组可调参数：隐层基函数中心、方差，以及输出单元的权值。
- 可以根据先验知识实现确定径向基函数的个数、中心、宽度等参数，也可以采用聚类分析等方法帮助确定。权值可以采用梯度下降法学习。

径向基函数网络（RBF）

- **RBF网络的作用**：
  - 把网络看成对未知函数$f(x)$的逼近器。一般任何函数都可表示成一组基函数的加权和，这相当于用隐层单元的输出函数构成一组基函数来逼近$f(x)$。
  - 在RBF网络中，从输入层到隐层的基函数输出是一种非线性映射，而输出则是线性的。这样，RBF网络可以看成是首先将原始的非线性可分的特征空间变换到另一空间(通常是高维空间)，通过这一变换使在新空间中线性可分，然后用一个线性单元来解决问题。

反馈型神经网络--Hopfield网络

- Hopfield网络是一种反馈网络。这种神经网络的特点是，输入信号作用于神经节点上，各个节点的输出又作为输入反馈到各个节点，形成一个动态系统，当系统稳定后读取其输出。

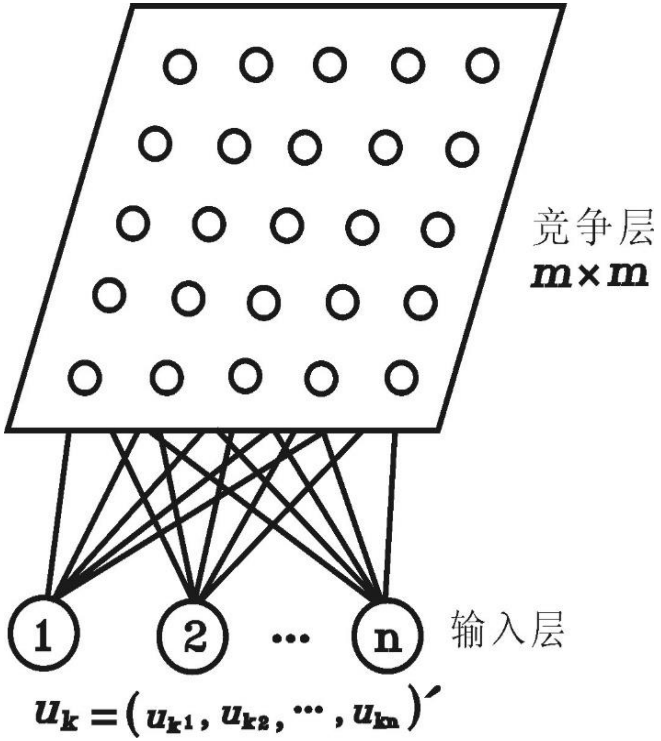


自组织特征映射神经网络（SOM）

- 人脑的记忆不是神经元与记忆模式的一一对应，而是一群神经元对应一个模式。
- 生理实验表明，某一外界信息所引起的兴奋刺激并不只针对一个神经细胞，而是对以某一个神经元为中心的一个区域内各神经元的兴奋刺激，并且刺激强度以区域中心为最大，随着与中心距离的增大，强度逐渐减弱，远离区域中心的神经元反而受到抑制。
- 自组织特征映射(Self-Organizing Feature Map，SOM)人工神经网络可以很好地模拟人类大脑的功能区域性、自组织特性及神经元兴奋刺激规律。

SOM网络模型及原理

- 在竞争学习网络中，所有的神经元节点通常排列在同一个层次上，没有反馈连接，但是神经元之间有横向的连接或相互影响。在学习时，通过神经元之间的竞争实现特定的映射。
- SOM网络是一种典型代表，其模型结构由输入层和输出竞争层组成，输入层与输出层之间是全互连的。有时输出层各神经元之间还有侧抑制连接。



模式识别

第三章：非线性判别函数

北京航空航天大学 人工智能研究院
Institute of Artificial Intelligence， Beihang University

杨鸿宇

2022年秋季学期

Fall 2022